Manual de desarrollo en Atlax Renpy

Versión 1.1

Autor:

Atlantox

Desarrollador de Campanella Studios

Índice

Contenido

Índice 2

Introducción 4

Requisitos 4

- Software y Hardware 4
- Estructura de una novela visual 4
  - Novela Visual 5
  - Ruta 5
  - Capítulo 5
  - Escena 5
  - Diálogo 5
  - Decisiones 5
  - Consecuencias 5
  - Puntuación 6

Flujo de trabajo 7

- ¿Cómo funciona Atlax Renpy? 7

Instalación 8

Configuración 9

- Definición del punto de inicio 10
- Definición de rutas base 10
- Definición de fondos 11
- Definición de personajes 12
- Definición de los idiomas 14
- Definición de configuraciones técnicas 15

Archivos de control 16

- Estructura 18
- Columna Key 18
- Columna Background 19
  - Tipos de transición de fondos 20
- Columna Music 21
- Columna Sound 21
- Columna Single effect 22
  - Efectos simples 23
- Columna Continuous effect 23

Efectos que interactúan con el fondo 24
Efectos que colocan una imagen encima del fondo 24
Columna Events 25
Posiciones y valores 29
Eventos de aparición de personajes 30
Eventos de personajes en pantalla 30
Animaciones constantes 32
Columna Delay 33
Columna Emisor 35
Columnas de Idioma 35
Límite de caracteres 36
Modificadores de texto 36
Orden de ejecución 37
Finalización de escena 37
Finalización lineal 38
Finalización con decisión 40
Finalización por condición 41
Scripts personalizados 45
Finalización mostrando los créditos 47
Finalización a la pantalla de título 47
Reproducir video tras terminar una escena 48
Limpieza del juego 48
Ejemplos 48
Encriptar archivos de control 49
Testing 50
Personalización 51
Animaciones personalizadas 51
Balanceo de novela visual 53
Derechos 55
Referencias 55

Introducción

Atlax Renpy es un motor de videojuegos del género novela visual clásica que utiliza como base el motor **Renpy** en su versión 8.1.3. Este motor permite el desarrollo de videojuegos con mínimos conocimientos informáticos y de programación. El motor está hecho para ser una fábrica de novelas visuales alimentada únicamente por **archivos de control** fácilmente manipulables y transportables.

Gracias a la filosofía de trabajo de **Atlax Renpy**, este ofrece una serie de ventajas enfocadas a dar facilidades a los escritores para poder plasmar, observar y corregir lo que están escribiendo tal cual como lo vería un usuario final del videojuego. Estas ventajas hacen posible que diversos escritores puedan crear sus respectivas historias de una misma novela simultáneamente y así desarrollar un videojuego sólido.

Requisitos

Para utilizar **Atlax Renpy** es obligatorio que aquellos miembros del equipo destinados a escribir dominen ciertos conceptos, además de cumplir con los requisitos de software y hardware.

Software y Hardware

- **Computadora:** Con sistema operativo Windows o cualquier otro compatible con los siguientes softwares.
    - **Editor de texto:**
        - Visual Studio Code (recomendado)
        - Bloc de notas
        - Sublime Text
        - Notepad
    - **Herramienta para abrir y editar archivos CSV:**
        - Excel (recomendado)
        - LibreOffice (Para usuarios de Linux)
        - Visual Studio Code
    - **Renpy:** En su versión 8.1.3. (Es posible que sea compatible con versiones posteriores)

Estructura de una novela visual

Seguramente tengas algunos conocimientos sobre las novelas visuales, ya sean conocimientos técnicos o de usuario final, en esta sección se va a desglosar la estructura y funcionamiento que sigue **Atlax Renpy** para mantener el control de todo lo que sucede internamente. Iremos de lo macro a lo micro.

Novela Visual

Una novela visual es una obra conjunta que podría ser definida como leer una historia. Esta puede contener decisiones y, por ende, distintas rutas o caminos con distintos finales. Sin embargo, siempre tienen el mismo punto de inicio, es decir, el primer capítulo o prólogo.

Ruta

Existen novelas visuales que son lineales, es decir, que la historia siempre será la misma, podríamos definirlas como "Novelas visuales de una sola ruta, un solo inicio y un solo final". Sin embargo, no es el caso de todas, usualmente las novelas visuales tienen rutas y cada ruta representa una posible historia para el protagonista, además de poder contener más de un final.

Capítulo

Un capítulo contiene un conjunto de **escenas** que engloban un arco, temporada o desafío en la historia de la novela. Un capítulo podría ser por ejemplo un semestre o periodo académico.

Escena

Una escena es una cadena de **diálogos** ordenados que van unos detrás de otros presentando un escenario al jugador. Las **escenas** tienen un inicio y un final, final que en ocasiones puede conducir a una o más **escenas**, ya sea por una decisión planteada al jugador o una condición basada en acciones anteriores.

Diálogo

Un **diálogo** es la pequeña pieza de una escena cuyo objetivo es sumergir al jugador una situación específica en pantalla. Un **diálogo** no solo se conforma por el texto mostrado al jugador, sino que es el conjunto de acciones y eventos que suceden y transmiten al jugador lo que está sucediendo. Un diálogo se compone de lo siguiente:

- Texto a mostrar en pantalla
- Sonidos y músicas
- Escenarios o fondos
- Despliegue de imágenes
- Transiciones
- Efectos visuales
- Aparición de personajes
- Acciones y movimientos de personajes

Decisiones

Una decisión es una situación que se plantea al jugador en la que debe escoger una opción de entre varias, dependiendo de la opción que escoja el jugador, habrá cambiado el rumbo de la historia (no siempre).

Consecuencias

Una consecuencia no es más que los resultados de las acciones del pasado, esto nos permite bifurcar la historia indirectamente sin que haya una intervención del jugador.

Las consecuencias no solo se basan en decisiones del pasado, sino también en escenas vistas o **puntuaciones**.

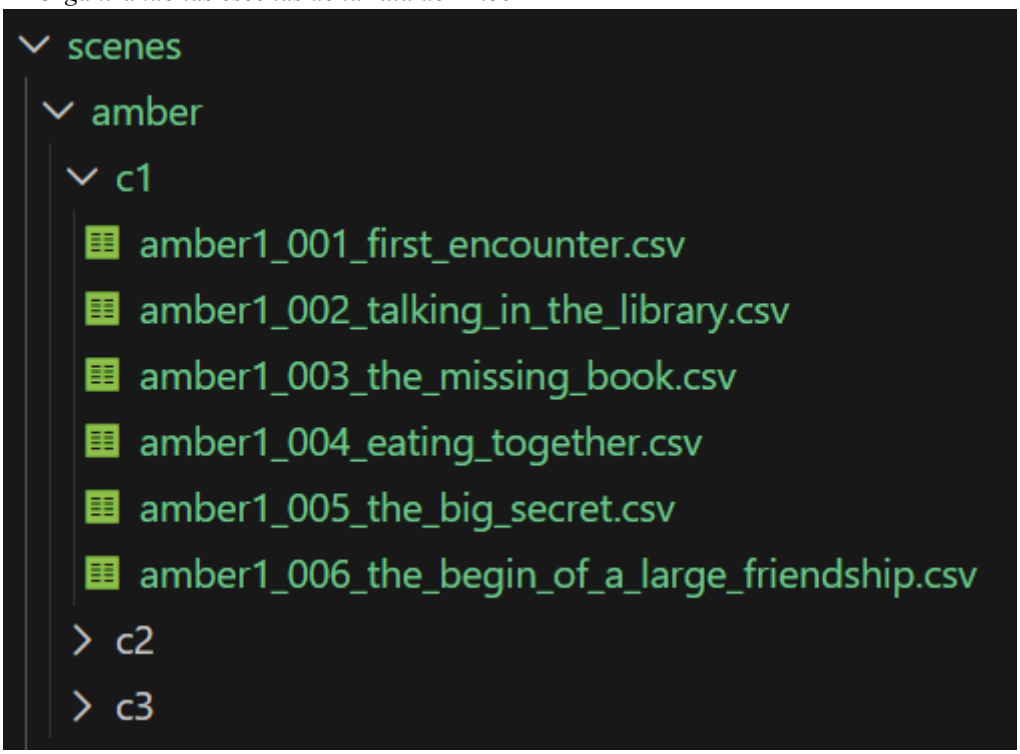
Puntuación

Los puntos son datos numéricos que se asocian a un nombre, estos puntos pueden aumentar y disminuir dependiendo de las decisiones que tome el jugador. Estos pueden tener cualquier nombre, amistad, calificación, fuerza, inteligencia, edad, etc. Los puntos nos sirven para crear bifurcaciones en la historia comparándolo con valores establecidos, por ejemplo, puedes hacer que al final de la historia, si el jugador alcanzó 10 o más puntos de amistad, este obtenga el final bueno, de lo contrario un final malo.

Es necesario que logres interiorizar y comprender los conceptos antes mencionados ya que estos serán la base de lo que sigue a continuación en este manual. Por supuesto que existen muchas otras posibles definiciones a estos conceptos, sin embargo, nos vamos a enfocar en lo que se necesita saber para utilizar **Atlax Renpy**.

Un buen ejercicio sería dividir tu novela visual en rutas, capítulos y escenas, para así, ir armando un sistema de carpetas organizadas. Mi recomendación es asignar una carpeta a cada ruta y desde ahí, una para cada capítulo y dependiendo de la complejidad de este, agregar subcarpetas para las escenas relacionadas. Por supuesto, todo dentro de la carpeta *scenes*.

Organizando las escenas de la ruta de Amber



Flujo de trabajo

¿Cómo funciona Atlax Renpy?

El motor funciona gracias a los **archivos de control**, que en esencia cada uno representa una **escena** distinta, el motor lee estos archivos y los ejecuta línea por línea mostrando al jugador los diálogos, fondos, personajes, efectos y demás.

Toda novela visual tiene un inicio, así que tendrás que definir un primer **archivo de control**, ya sea un prólogo o capítulo 1, una vez definido, **Atlax Renpy** va a leer ese primer archivo de control y comenzará a mostrar los **diálogos** como de costumbre, uno a uno, click a click.

Cuando el motor llega al final de una **escena**, esta puede tener distintas formas de terminar, puede terminar de forma lineal, con una decisión, con una bifurcación o incluso mostrando los créditos finales.

Usualmente una **escena** terminará conduciendo al jugador a otra **escena** distinta, y esa escena a otra más, y así sucesivamente creando una **cadena** de **escenas** que se ejecutan de manera ordenada una después de otra.

Haciendo uso de las **decisiones** podemos crear **ramificaciones** y añadir o quitar **puntuaciones.** Si tu novela visual posee **decisiones** y **consecuencias**, se aconseja encarecidamente mantener un esquema con todas las decisiones, bifurcaciones y puntos, de esta forma podrás tener una idea sobre la complejidad de los **caminos posibles** de tu historia.

Si tienes dudas sobre cómo hacerlo de manera efectiva, yo mismo he desarrollado una herramienta para definir qué tan compleja son las decisiones de tu novela visual, se llama **Visual Novel Technical Assitant** y en la sección de **balanceo de novela visual** se explicará cómo usarlo.

Una de las principales ventajas que nos ofrece **Atlax Renpy** es permitir que distintos escritores trabajen de manera simultánea con sus propias **escenas, capítulos y rutas** ¿Lo mejor? Serán capaces de testear y ver con sus propios ojos las **escenas** que escriban, viéndolas tal cual las vería un usuario final.

Recomendación: Escribe tu novela utilizando un archivo Word utilizando Google Drive, procurando que cada línea de texto sea un diálogo.

Escribiendo un solo salto de línea para cada diálogo.

Me defiendo como puedo. Sus armas son cortas pero su sincronización es casi perfecta.

Intento mantenerlos a raya aunque solo estoy retrasando lo inevitable, a este ritmo voy a perder.

Me quedo sin espacio y termino arrinconado defendiéndome de sus cortes y puñaladas con mis guantallas.

Ámber: -¡Dejen en paz a nuestro héroe!

Ámber hace aparición atravesando la nube de polvo apuntando hacia al frente con su estoque emitiendo un aura desafiante y decidida.

Justo detrás de ella, el resto del equipo aparece abriéndose paso por la nube arremetiendo contra los enemigos restantes.

¿Has terminado de escribir tu novela? ¿Has definido ya las rutas, decisiones, consecuencias y puntuaciones de tu historia? Ha llegado la hora de instalar **Atlax Renpy** y traducir los textos de tu novela en **archivos de control**.

Instalación

Puedes optar por ver el siguiente video explicativo sobre **Atlax Renpy** y su instalación: XXX

Sin embargo, en esta sección de todas formas se explicará cómo instalarlo.

1. Descargar el instalador de **Atlax Renpy** desde el repositorio oficial: https://github.com/Atlantox/atlax-renpy/releases
2. Descargar e instalar **Renpy** en su versión 8.1.3 (Es posible que **Atlax Renpy** siga siendo compatible con versiones posteriores)
3. Crear un nuevo proyecto con **Renpy**.
4. Mover el instalador (.exe) de **Atlax Renpy** a la carpeta raíz del proyecto creado. El instalador debe estar en el mismo nivel que la carpeta *game*.
5. Ejecutar el instalador. Esto creará todo el sistema de carpetas además de los archivos necesarios para su correcto funcionamiento además de este manual y un par de ejemplos.
6. La instalación ha sido terminada, ya puedes iniciar el desarrollo de tu novela visual.

Es importante que al momento de desarrollar tu novela visual consideres utilizar un repositorio para respaldar tu proyecto. Mi recomendación es utilizar **Github**: https://github.com/ se puede llegar a hacer algo confuso como utilizarlo así que te recomiendo buscar tutoriales para respaldar de manera segura los avances que vayas realizando.

Por supuesto, también puedes optar por utilizar respaldos manuales usando Google Drive o Mega, no obstante, mi recomendación siempre será usar **Github**. El editor de texto **Visual Studio Code** tiene una muy buena integración con **Github**.

Configuración

Una vez ya instalado **Atlax Renpy**, lo primero será acudir al archivo de configuración, este estará dentro de la carpeta *game* y su nombre será *config.csv*. Los archivos de formato .CSV (Comma-Separated Values) pueden ser abiertos con **Excel** (altamente recomendado) o con algún editor de texto como bloc de notas o **Visual Studio Code**.

Si por alguna razón encuentras el archivo pero este no termina en ".csv", busca en la configuración de tu sistema operativo y habilita la visibilidad de los formatos de los archivos, de esta manera podrás ver el ".csv" al final del archivo *config.csv*.

Importante: Si abres un archivo .CSV con Excel, procura guardarlo en el formato CSV UTF-8.

Al abrir el archivo de configuración usando Excel, encontrarás una tabla similar a la siguiente.

Archivo de configuración. Ubicado en game/config.csv

	A	B	C	D	E	F	G
1	Key	Value1	Value2	spanish	english	chinese	portugues
2	#						
3	# The first CSV scene / custom scene						
4	#						
5	begin						
6	#						
7	# Base path configs, preferably not touch						
8	#						
9	path_background	images/backgrounds/					
10	path_sound	sounds/					
11	path_music	music/					
12	path_scene	scenes/					
13	path_displayable	images/displayables/					
14	#						
15	# Background define						
16	#						
17	background	background name					
18	#						
19	# Character define						
20	#						
21	character	character key	character name color	name in spanish	name in english	name in chinese	name in portugues

El archivo de control se compone por columnas, estas son *Key, Value1 y Value2*. Las demás columnas corresponden a los idiomas admitidos por nuestra novela visual, siéntete en libertad de reorganizar o agregar nuevos idiomas, también puedes quitar los que no necesites.

El primer idioma será considerado el idioma **por defecto** de la novela.

Importante: Una vez definas los idiomas, procura escribirlos siempre en minúscula y respetar el orden en que están, ya que estos idiomas serán utilizados en varios sitios del motor.

El propósito de cada columna es el siguiente:

- Siempre que una fila comience con el carácter "#", la fila será ignorada y se pasará a la siguiente.
- Columna Key: Esta explica qué elemento se está definiendo, estos pueden ser: personajes, fondos, inicio de la novela y rutas base (base_paths).
    - Se recomienda no tocar las rutas base.
- Columnas Value1 y Value2: Establecen el o los valores del elemento que se esté definiendo.
- Columnas de idioma: Se usan para definir las variaciones de nombres de personajes dependiendo del idioma. Si un idioma no tiene una variación de nombre, utilizará la primera (la variación por defecto).

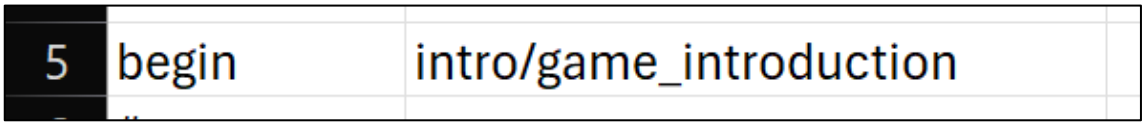
A continuación, se detallará como definir correctamente cada uno de los posibles elementos.

Definición del punto de inicio

El *Key* correspondiente al punto de inicio de la novela es el "begin", lo encontraremos aproximadamente en la fila 5 y esta solo utilizará la columna *Value1*.

En la columna Value1 escribiremos la ubicación y nombre del nombre del **archivo de control** o el label del **script personalizado** desde la cual comenzará la novela. Por ejemplo, si nuestra escena de inicio está ubicada en la carpeta *intro* y se llama *game_introduction.csv*, nuestro *begin* se vería así.

Ejemplo de punto de inicio al archivo ubicado en game/scenes/game_introduction.csv



Siempre que se haga referencia a un archivo de control, escribe solamente su nombre, no hace falta que escribas *.csv*

Por otro lado, si el inicio de nuestra novela es un **script personalizado**, colocaremos el nombre del **label** de nuestro script. Sin embargo, este asunto de los **scripts personalizados** será tratado más adelante con precisión.

Definición de rutas base

El *Key* correspondiente a una definición de ruta base comienza por "*path_"* y son 5 rutas base en total, lo encontraremos aproximadamente por la fila 9 hasta la fila 13 y solo utilizará la columna *Value1.*

Las rutas base son las rutas dentro de la carpeta *game* desde las cuales **Atlax Renpy** buscará los recursos asociados, es decir, los sprites, músicas, sonidos y demás. se recomienda encarecidamente no cambiar ninguno de estos valores.

Definiendo las rutas base en el archivo de configuración

6	#	
7	# Base path configs, preferably not touch	
8	#	
9	path_background	images/backgrounds/
10	path_sound	sounds/
11	path_music	music/
12	path_scene	scenes/
13	path_displayable	images/displayables/

path_background	La ruta de los fondos a mostrar.
path_sound	La ruta de los efectos de sonido.
path_music	La ruta de las canciones y músicas.
path_scene	La ruta donde se almacenarán todos los archivos de control (con su respectiva jerarquía de carpetas).
path_displayable	La ruta de las imágenes que se mostrarán en pantalla (distinto a los fondos y personajes).

Definición de fondos

El *Key* correspondiente a una definición de fondo comienza por "*background"*, lo encontraremos aproximadamente por la fila 17 y solo utilizará la columna *Value1*.

Los fondos son los escenarios que se muestran detrás de los personajes como por ejemplo un salón, un parque o un cielo estrellado.

Debes guardar los fondos en la carpeta correspondiente al *path_background*, por defecto será *game/images/backgrounds*. Deben ser en formato .png y preferiblemente tener la misma resolución. El tamaño recomendado es 1920x1080 pixeles.

Para definir un fondo simplemente debemos escribir en la columna *key* "background" y en la columna *Value1* escribir el nombre del archivo respetando los espacios. Por ejemplo, si deseamos implementar un fondo cuyo archivo es "dark forest.png", la definición sería la siguiente.

Definiendo el fondo cuyo nombre es "dark forest.png"

31	background	dark forest

Si revisamos el archivo de configuración notaremos que ya hay algunos fondos definidos, estos son *flash* y *blackout*. Estos fondos son usados por el motor así que procura no borrarlos.

Los fondos pueden estar guardados en cualquier parte del proyecto, con darle nombre, el motor se encargará de buscar el fondo esté donde esté, esto nos permite organizar los fondos en subcarpetas como mejor nos convenga.

Definiendo todos los fondos de nuestra novela

14	#		
15	# Background define		
16	#		
17	background	flash	
18	background	blackout	
19	background	training field	
20	background	smoke	
21	background	forest	
22	background	suffocation	
23	background	city	
24	background	mall	
25	background	puerkokun	
26	background	house inside	
27	background	house front	
28	background	smoke	
29	background	forest	
30	background	dark forest	
31	background	dark forest flash	
32	background	factory	

Realizaremos este procedimiento para cada fondo de nuestra novela visual, al final nos quedará algo como la imagen de la izquierda.

Definición de personajes

El *Key* correspondiente a una definición de personaje comienza por "*character"*, lo encontraremos aproximadamente por la fila 22 y utilizará tanto la columna *Value1* como *Value2.* Además, esta utilizará también las columnas de idiomas.

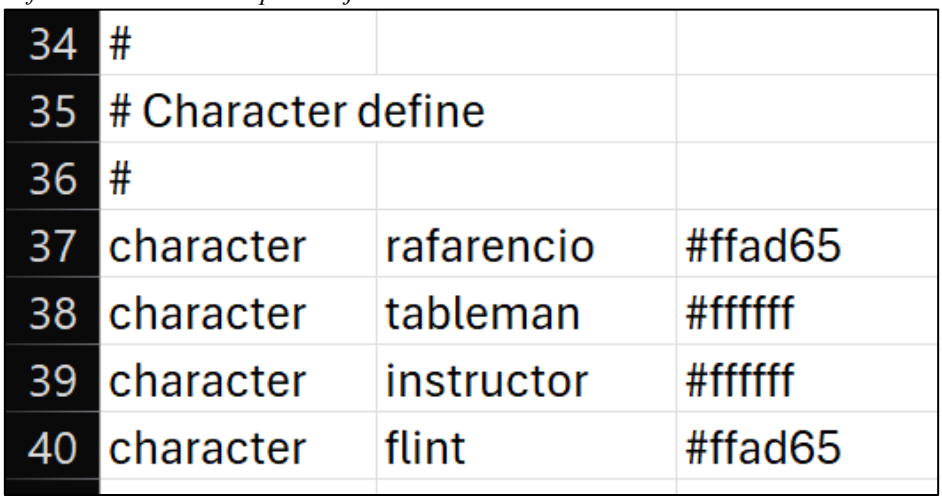
En la columna *Value1* colocaremos el **nombre clave** del personaje, mientras que en la columna *Value2* escribiremos el color del nombre del personaje cuando este hable, este color debe ser escrito en código hexadecimal.

Importante: El nombre clave debe estar en minúscula.

Si desconoces lo que es un código hexadecimal, es un conjunto de caracteres que se usan para representar un color, puedes visitar el siguiente sitio web para descubrir los códigos hexadecimales de los colores que requieras https://htmlcolorcodes.com/es/selector-de-color/

Estos códigos comienzan con un “#” y le siguen 6 dígitos que pueden ir del 0 al 9 y de la A hasta la F.

Definiendo una serie de personajes



34	#		
35	# Character define		
36	#		
37	character	rafarencio	#ffad65
38	character	tableman	#ffffff
39	character	instructor	#ffffff
40	character	flint	#ffad65

Si por ejemplo un personaje se llama Katherine y quieres que su nombre aparezca en color rosado, para definirlo tendrías que escribir la siguiente línea:

Definiendo al personaje Katherine



41	character	katherine	#fb51f9

Ahora vamos con un aspecto interesante sobre los personajes, además de definir el **nombre clave** y el color, tendremos que definir el nombre del personaje en los distintos idiomas. En el caso de Katherine, simplemente debemos escribirlo una sola vez, ya que, este personaje tendrá siempre el mismo nombre en todos los idiomas.

Por lo tanto, la definición completa de Katherine sería la siguiente.

Definición completa de Katherine, personaje cuyo nombre es el mismo en todos los idiomas.



42	character	katherine	#fb51f9	Katherine

Si bien es cierto que generalmente los nombres de los personajes no van a variar dependiendo del idioma, existen algunos que sí, por ejemplo, un mesero. Si definiésemos al personaje bajo el nombre “Mesero”, dicho nombre aparecería en todos los idiomas, no tendría sentido que al jugar en inglés apareciese “Mesero”, este debería decir “Tableman”, lo mismo para otros idiomas.

En este caso, como el nombre del personaje puede variar según el idioma, utilizaremos el resto de columnas de idioma, por lo tanto, nuestro personaje mesero se definiría de la siguiente manera.

Definiendo al personaje "Mesero" y colocando su nombre dependiendo del idioma.

38	character	tableman	#ffffff	Mesero	Tableman	服务员	garçom

Es necesario que entiendas que el personaje puede tener múltiples variaciones de nombre, sin embargo cuando hagamos referencia a este personaje, lo haremos mediante su **nombre clave,** en este caso "tableman" (definido en la columna *Value1*).

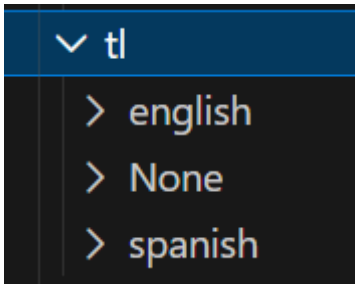
De esta manera siempre que el personaje "tableman" hable, su nombre se colocará en pantalla utilizando su idioma correspondiente. El motor siempre intentará mostrar el nombre del personaje según el idioma actual, si este no tiene variación en dicho idioma, mostrará el nombre **por defecto**.

Allí tienes que definir TODOS los personajes que hablen y aparezcan en tu novela.

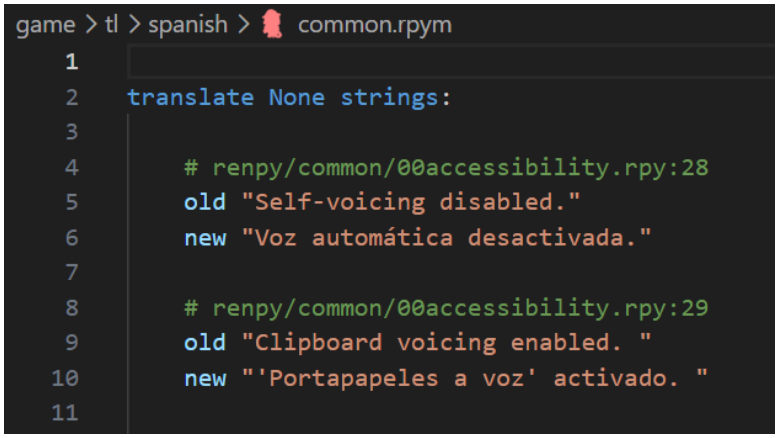
Definición de los idiomas

Cuando crees un proyecto en **Renpy**, este generará una carpeta llamada *tl*, esta contendrá los idiomas y traducciones permitidos. **Renpy** hace referencia al idioma por defecto bajo el nombre de "*None*", es necesario que copies y pegues esa carpeta bajo el nombre del idioma por defecto de tu novela. Luego de eso, accede al archivo *common.rpym* y cambia "None" por "tu idioma por defecto".

Copiando la carpeta None y pegándola bajo el nombre de "spanish"



Abriendo el archivo common.rpym dentro de spanish usando Visual Studio Code



Cambiando "None" por el idioma por defecto "spanish"

```
game > tl > spanish > common.rpym
1
2   translate spanish strings:
3
4       # renpy/common/00accessibility.rpy:28
5       old "Self-voicing disabled."
6       new "Voz automática desactivada."
7
8       # renpy/common/00accessibility.rpy:29
9       old "Clipboard voicing enabled. "
10      new "'Portapapeles a voz' activado. "
11
```

En caso de que requieras agregar más idiomas, procura ir al ejecutable de **Renpy** y crear la traducción. Al crear la traducción **Renpy** generará la carpeta correspondiente en la carpeta *tl*. Recuerda nombrar los idiomas siempre en minúscula.

El archivo *common.rpym* contiene todas las traducciones de todos los textos del juego. Existen softwares que traducen tu proyecto en **Renpy** automáticamente que pueden encontrarse por internet. Sin embargo, también puedes optar por usar mi repositorio de lenguajes en **Renpy**.

Actualmente este repositorio contiene las traducciones de inglés a español y viceversa. Puedes encontrarlo aquí: https://github.com/Atlantox/renpy-languages

Definición de configuraciones técnicas

Si eres un escritor puedes saltarte esta sección. En caso de que quieras cambiar previamente algunos de los valores predeterminados de **Renpy** puedes hacerlo en el archivo *customConfig.rpy* ubicado en la carpeta *game/definitions.* Allí podrás definir todas las configuraciones extras que quieras realizar en **Renpy**, evidentemente estas configuraciones deben hacerse en lenguaje **Renpy**.

Por defecto **Atlax Renpy** centra la caja de texto con el nombre del emisor entre otras configuraciones. Si decides abrir el archivo, dentro encontrarás algo como esto.

El contenido del archivo game/definitions/customConfig.rpy

```
1   define config.enable_language_autodetect = False
2   define config.has_sync = False
3   define config.fadeout_audio = 3
4
5   define gui.name_xpos = 0.5
6   define gui.name_xalign = 0.5
7
8   image bg main_menu_background = gui.main_menu_background
9
```

Una vez definida la configuración e idiomas de nuestro proyecto, podemos pasar a la creación de los **archivos de control.**

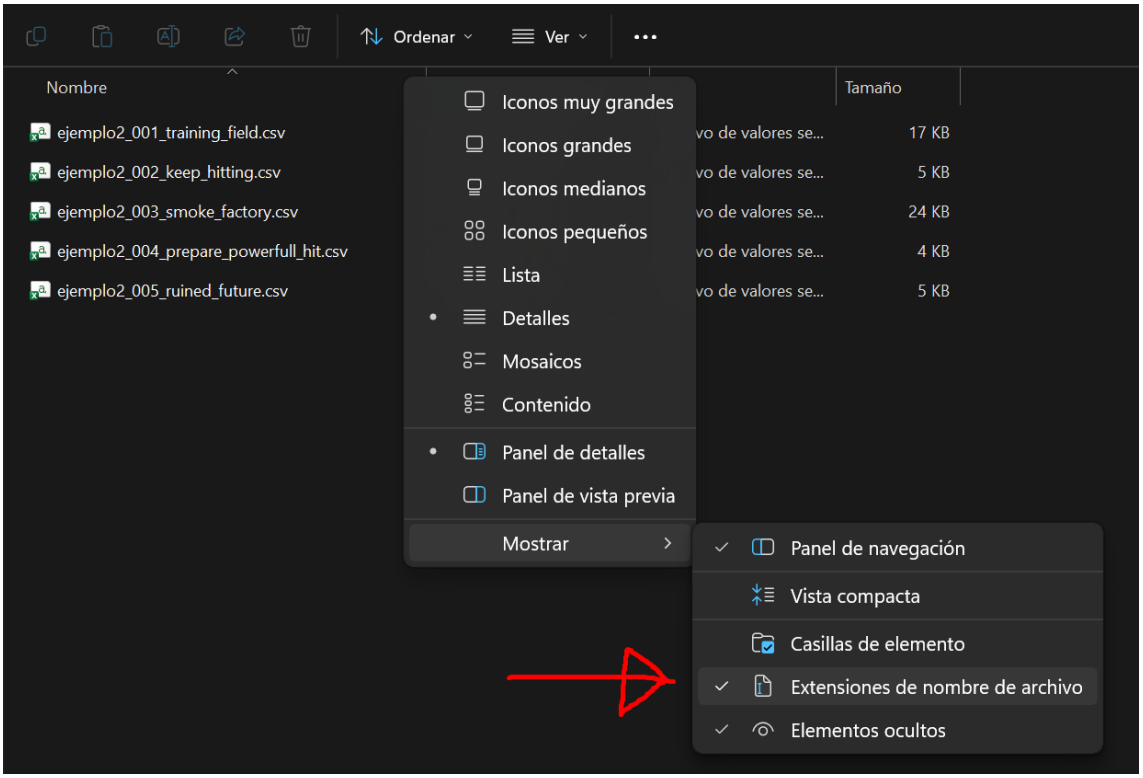
Archivos de control

Un archivo de control representa literalmente una **escena** en su totalidad, su contenido consiste en múltiples diálogos representados en líneas de texto estructurado y ordenado. El formato de estos archivos es .CSV, es decir, un archivo separado por comas, específicamente por puntos y comas (**;**). Si bien es cierto que este tipo de archivo se puede abrir con bloc de notas, se recomienda encarecidamente utilizar Microsoft Excel para mayor comodidad.

Por defecto estos archivos deben ser guardados en la carpeta *scenes*.

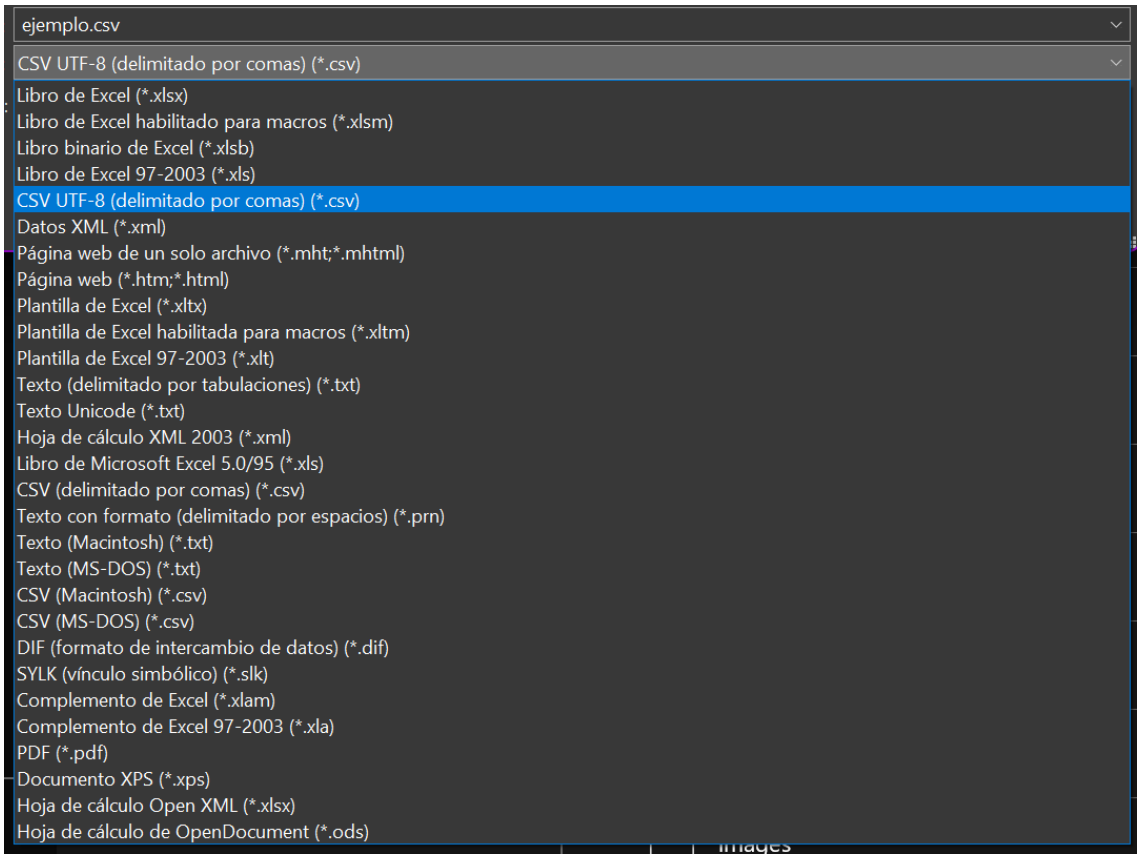
Crear un archivo de control es muy sencillo, tenemos dos formas de hacerlo, la forma "complicada" es creando un archivo de texto en nuestro computador y colocarle al final .csv, llamémoslo "ejemplo.csv". Es posible que al abrirlo se ejecute el bloc de notas, para ello tenemos que ir a la configuración de nuestro sistema operativo para que nos permita ver las extensiones o formatos de los archivos, una vez cambiado debemos cambiar ".txt" por ".csv".

Habilitando la visibilidad de las extensiones de los archivos en Windows 11.



La forma sencilla es simplemente abriendo Excel, ir a Guardar Como y entre los tipos de archivo escoger **CSV UTF-8**.

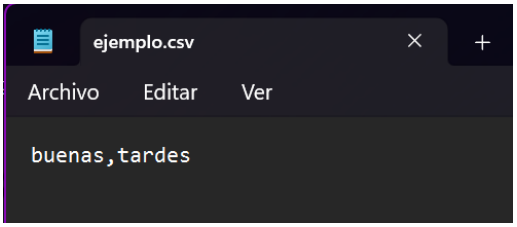
Guardando un archivo como CSV UTF-8 usando Microsoft Excel.



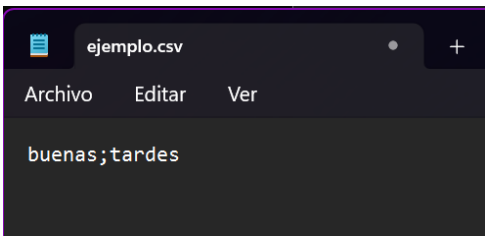
Si tienes dudas puedes optar por simplemente copiar y pegar alguno de los **archivos de control** de ejemplo que proporciona **Atlax Renpy**, borrar su contenido y empezar desde allí.

Guardas el archivo y listo, sin embargo, se recomienda escribir cualquier cosa en su contenido y luego abrir el archivo usando bloc de notas para asegurarte de que las columnas se estén separando por puntos y comas. Si no lo están, cambia la coma por un punto y coma y listo.

Abriendo el archivo ejemplo.csv



Sustituyendo la coma por punto y coma.



Estructura

Los archivos de control no son más que archivos parecidos a tablas de **Excel** con columnas separadas por puntos y comas, lo más fundamental de estos archivos será la primera línea, es decir, la cabecera, esta nos dará los nombres de las columnas en las que vamos a trabajar nuestra novela visual, si abrimos con bloc de notas veremos algo como esto en la primera línea de texto.

Key;Background;Music;Sound;Single effect;Continuous effect;Events;Delay;Emisor**;**spanish;english

Si lo abrimos con Excel veremos algo así

Cabecera de un archivo de control.



	A	B	C	D	E	F	G	H	I	J	K
1	Key	Background	Music	Sound	Single effect	Continuous e	Events	Delay	Emisor	spanish	english

La manera en que trabajaremos en estos archivos es escribiendo los diálogos línea por línea describiendo cada uno de los eventos que suceden en pantalla.

A continuación, detallaremos cada una de las columnas, sin embargo, antes de ello hay que hacer algunas aclaraciones.

- Si un diálogo comienza con "//", ese diálogo será ignorado, úsalo para escribir comentarios en un archivo de control.
- Los números decimales deben ser escritos con puntos en vez de coma.
- Todas las **palabras clave** que escribas deben ser en minúscula.
- Notarás que en las explicaciones de algunos campos hay un valor entre paréntesis, ese valor representa su valor por defecto.
- Los ejemplos serán subrayados de esta forma.

Columna Key

Esta columna representa un identificador único que debe tener cada diálogo, ese identificador debe ser irrepetible dentro del mismo archivo de control. Aún así. se recomienda que tampoco se repita en otras escenas. Esto ayudará al momento de corregir errores ya que sabrás exactamente en qué línea está el problema.

El valor recomendado para colocar en esta columna **Key** es el siguiente: [ruta]_[capitulo]_[escena]_[dialogo], por ejemplo, si queremos describir el diálogo número 89 de la escena noche de paseo del capítulo 3 de la ruta Stefannie, el **Key** quedaría algo así.

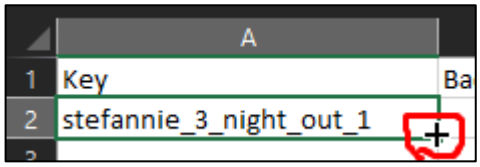
stefannie_3_night_out_89

El siguiente diálogo se escribiría igual pero con un 90 en vez de 89 y siempre empezando en 1.

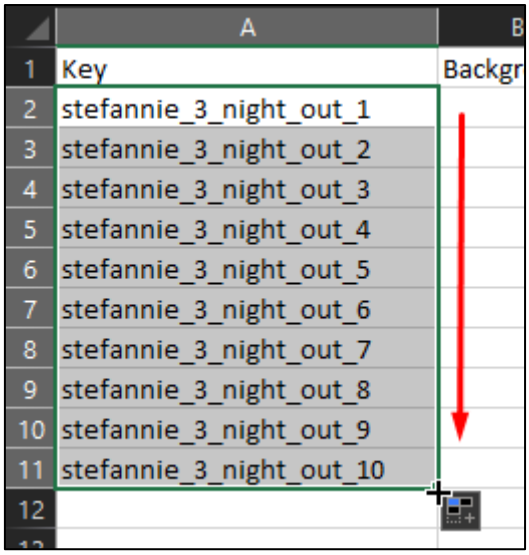
Debido a que puede llegar a ser demasiado tedioso escribir a mano todos los números, Excel nos brinda una ayuda, primero abrimos el archivo usando **Excel**,

escribimos en la primera fila (después de la cabecera) stefannie_3_night_out_1, acto seguido dirigimos el cursor a la esquina inferior derecha de la celda seleccionada y manteniendo el click, lo arrastramos hacia abajo hasta el final del archivo.

Seleccionando la primera celda



Manteniendo click y arrastrando hacia abajo



De esa manera Excel autocompletará las celdas y nos ahorrará escribir todos los números de cada diálogo.

Columna Background

En esta columna vamos a escribir los fondos que aparecerán en el juego, pudiendo controlar la manera en que se muestran. Deben estar en formato .png, tener una resolución de 1920x1080 pixeles y por defecto estar ubicados en *game/images/backgrounds*. Sintaxis:

Nombre del fondo : tipo de transición : parámetros de la transición

El nombre del fondo es el que hayamos colocado en la definición de fondos del archivo *config.csv*. Si colocamos el nombre del fondo sin más, por defecto el motor mostrará el fondo con un **Dissolve**, es decir, con un desvanecimiento del fondo actual y una aparición del siguiente fondo a lo largo de 3 segundos.

Tipos de transición de fondos

Nombre	Descripción	Parámetros
dissolve	El fondo actual se desvanece a la vez que el siguiente aparece	1. Los segundos que los fondos tardan en cambiar (3)
fade	La pantalla se funde a negro y luego muestra el siguiente fondo	1. Los segundos que tomará el fondo actual en desvanecerse (1)
		2. Los segundos de pantalla en negro tras fundirse el fondo anterior (0.5)
		3. Los segundos que tomará el fondo nuevo en aparecer (1)
pixel	El fondo actual se pixela hasta el color de mayor presencia, luego cambia de color y muestra el siguiente fondo haciendo el proceso inverso. (Se recomienda dejarlo por defecto)	1. La duración en segundos (3)
		2. La cantidad de pasos para llegar al color de mayor presencia (18)
pushup / pushright / pushdown / pushleft	El fondo actual se desliza hacia una dirección para dar lugar al nuevo fondo	1. La duración del desliz en segundos (1.5)
rup / rright / rdown / rleft	El fondo actual rota perpendicularmente hacia la dirección dada	1. La duración de la rotación en segundos (1.5)

school hall 3:dissolve:4 (El fondo school hall 3 aparece a lo largo de 4 segundos)

campus:fade:1:3:1 (El fondo campus aparece luego de un fundido a negro)

music room (Aparece con la transición por defecto)

Podemos hacer que en un mismo diálogo haya múltiples cambios de fondo, para ello debemos separarlos por comas. Los fondos se mostrarán en orden siguiendo las transiciones dadas. Por ejemplo, si el protagonista va del salón al comedor y se desea mostrar una serie de fondos consecutivos para transmitir el recorrido, haríamos algo como esto en la columna **Background**.

school hall 2, school stairs, school dining room

Suponiendo que el fondo actual es el salón de clases, el fondo cambiará al pasillo, luego a las escaleras y de último el comedor.

Estas transiciones se ejecutan una a una antes de mostrar los diálogos correspondientes, el jugador podrá dar click para saltarse las transiciones de un fondo a otro.

Recuerda que cada fondo mostrado de esta forma también puede ir acompañado con transiciones, si no se le coloca ninguna usará **Dissolve**.

Columna Music

La columna encargada de realizar los cambios de música en el juego, estos deben estar en la carpeta *game/audio/music* y además de poseer formato .wav. Hay que dejar claro 2 conceptos.

1. **Fade-in:** Tiempo en el que una canción pasa del silencio al 100%
2. **Fade-out:** Tiempo en el que una canción pasa de su volumen actual al silencio

Cuando el juego esté en silencio y coloquemos una música, este hará un **fade-in** de 3 segundos.

Si pasamos de una canción a otra, la que se está reproduciendo hará un **fade-out** de 3 segundos y la canción entrante un **fade-in** de 3. Su sintaxis es la siguiente:

Nombre de la musica : fade-in en segundos

El nombre de la canción que debes escribir es el del nombre del archivo de audio.

Para quitar la música y crear un silencio, simplemente debemos colocar un "*", por supuesto, también podemos acompañarlo del parámetro de **fade-in** que en este caso tomará el rol de fade-out.

while we speak:4 (La música actual hace el **fade-out** por defecto y la nueva **fade-in** de 4 segundos)

stride (Tanto la música actual y la entrante harán un **fade-out** y **fide-in** por defecto)

*:5 (La canción actual hace un **fade-out** de 5 segundos al silencio)

*:0 (La canción actual se detiene de golpe)

En el archivo ubicado en *game/definitions/customConfig.rpy* puedes encontrar y modificar la duración del **fade-in** y **fade-out** por defecto.

Columna Sound

Aquí escribirás los sonidos que se reproducen en cada diálogo. Deben estar en formato .wav y estar ubicados en la carpeta *games/audio/sounds*. Generalmente se reproducirán una vez, sin embargo, podemos hacer que se reproduzcan indefinidamente, también podemos reproducir varios sonidos a la vez. Sintaxis:

Nombre del sonido

Nombre del sonido 1, Nombre del sonido 2

Nombre del sonido : loop

Si separamos por comas varios sonidos, estos se reproducirán al mismo tiempo. Se pueden reproducir hasta 5 sonidos a la vez. Puedes cambiar esto en *game/managers/ConfigManager.rpy* en la variable *simultaneousSounds*

El nombre del sonido que se va a escribir debe ser el mismo que el archivo de audio, puedes utilizar espacios en los nombres. Las mayúsculas no se respetan, de todas formas, se recomienda escribirlo todo en minúscula.

Si colocamos el nombre del sonido sin más, este se reproducirá una sola vez. Para reproducir varios sonidos simultáneamente, debemos separarlos por comas. Si deseamos que el sonido se reproduzca constantemente debemos añadir el parámetro “loop”, es importante mencionar que hacer eso hará que el sonido se reproduzca indefinidamente hasta que volvamos a escribir la misma instrucción más adelante.

golpe (Reproduce golpe una sola vez)

golpe, latidos (Reproduce golpe y latidos una sola vez al mismo tiempo)

espadas chocando:loop (Reproduce espadas chocando indefinidamente hasta que se vuelva a escribir espadas chocando:loop en un diálogo posterior).

Se pueden colocar varios sonidos en loop y reproducir sonidos mientras otros sonidos están en loop, el límite depende de la variable *simultaneousSounds* con *ConfigManager.rpy.*

Columna Single effect

Son efectos visuales que suceden una única vez, como una sacudida de pantalla o flash repentino. Sintaxis:

Nombre del efecto : parámetros

Nombre del efecto 1 : parámetros, Nombre del efecto 2 : parámetros

Podemos colocar varios efectos en un mismo diálogo, estos se efectuarán uno a uno en el orden escritos.

Efectos simples

Nombre	Descripción	Parámetros
vpunch / hpunch	El fondo de pantalla se mueve en horizontal (hpunch) o vertical (vpunch) simulando con golpe de una intensidad dada	1. La intensidad del golpe. Ejemplo 10 un golpe sutil, 60 un golpe fuerte y 100 un golpe exagerado. (20)
Nombre de un fondo	El juego muestra unos parpadeos mostrando un fondo de pantalla una cantidad de veces a lo largo de una cantidad de segundos.	1. Los segundos que tarda el fondo en mostrarse y quitarse cada ciclo (0.1)
		2. La cantidad de ciclos (veces) que el fondo se va a mostrar (4)
		3. El nivel de opacidad de la imagen siendo 0 transparente y 1 visible (1)

vpunch:50 (Ocurre un golpe vertical con una intensidad fuerte)

hospital:0.1:3 (Muestra el fondo hospital 3 veces mostrando el fondo 0.1 segundos cada vez simulando una especie de flashback)

flash:0.25:1 (Muestra un fondo blanco una única vez durante 0.25 segundos simulando un destello repentino)

blackout:0.25:1:0.5 (Muestra un fondo negro una única vez durante 0.25 segundos simulando un breve corte de luz o parpadeo, el fondo negro se mostrará con opacidad del 0.5 así que el fondo actual será medianamente visible)

Columna Continuous effect

Es similar a los **Single Effects**, solo que esta vez es un efecto que sucede constantemente en pantalla y persiste indefinidamente hasta que se vuelva a llamar. Se pueden dividir en dos, aquellos efectos continuos que interactúan con el fondo actual, y aquellos que colocan una imagen por encima. La sintaxis es la misma que la de los **Single Effects**.

Nombre del efecto : parámetros

Nombre del efecto 1 : parámetros, Nombre del efecto 2 : parámetros

Efectos que interactúan con el fondo

Nombre	Descripción	Parámetros
blur	El fondo actual se vuelve borroso	1. La intensidad de la borrosidad. Siendo 100 una gran borrosidad. (15)
hwarp / vwarp	El fondo actual se estira vertical (vwarp) u horizontalmente (hwarp)	1.El factor de estiramiento. En 1 no se estira nada, en 2 el fondo duplica su tamaño horizontal o verticalmente. (1.1)
rotate	El fondo actual rota una cantidad de grados dada. No se recomienda usar mucho ya que deja ver el fondo trasparente por detrás	1. Los grados que el fondo es girado (45)

blur:30 (Una borrosidad considerable)
vwarp:1.5 (El fondo se estira verticalmente 1.5 veces su alto)
rotate:30 (El fondo rota 30 grados hacia la derecha en sentido de las manecillas del reloj)

Estos efectos son perfectamente acumulables, puedes colocarlos en diálogos separados o ponerlos en el mismo separando los efectos por comas. Para removerlos, tienes que escribir el nombre del efecto sin parámetros.

blur:10 (Poca borrosidad)
blur:20 (La borrosidad aumenta)
blur:10 (La borrosidad disminuye)
blur (Se elimina la borrosidad por completo)

Efectos que colocan una imagen encima del fondo

Nombre	Descripción	Parámetros
Nombre del fondo o imagen	Muestra una imagen por encima del fondo y los personajes en pantalla, se puede controlar su nivel de opacidad	1. La opacidad, donde 1 es la imagen normal y 0 es la imagen totalmente invisible. (1)

Podremos colocar fondos de pantalla encima del fondo actual, para que esto suceda solo basta escribir el nombre del fondo tal cual haya sido definido en *config.csv*. También podremos controlar su nivel de opacidad para que se vea transparente.

Un efecto interesante que podemos lograr es haciendo un fondo que consista en líneas rojas en el borde de la pantalla para simular un efecto de sofocación o sufrimiento, llamemos este fondo “suffocation”, la manera de mostrarlo sería la siguiente.

suffocation:0.5 (Muestra el fondo suffocation el cual será medio transparente)

Para que la imagen deje de mostrarse, entremos que escribir el nombre de la imagen en un diálogo posterior, en este caso, “suffocation”.

Es muy probable que en nuestra historia deseemos mostrar elementos en pantalla, ya sea un artilugio visto de cerca, un dibujo o plano. Para esto debemos primero colocar el archivo de dicha imagen en *game/images/displayables*, y acto seguido, escribir el nombre del archivo. El objeto aparecerá en pantalla hasta que se vuelva a escribir el nombre del archivo. También podremos controlar la opacidad de esta imagen.

family picture (Muestra una foto de una familia en pantalla)

Las imágenes deben estar en formato .png y se debe tener en cuenta la resolución que estas tienen ya que de ese tamaño se mostraran las imágenes, es decir, si intentas mostrar en pantalla una imagen de resolución 1920x1080, esta taparía toda la pantalla.

Si deseas mostrar un objeto, la resolución de este debe ser acorde a cómo quieres que se vea en pantalla, si el objeto tiene una resolución de 300x600, se mostrará en ese tamaño centrado en la pantalla.

Los nombres de los archivos a mostrar deben estar en minúscula.

Cuando escribas el nombre de una imagen, el motor priorizará encontrar un fondo con ese nombre, si no lo encuentra buscará entonces en *game/images/displayables.*

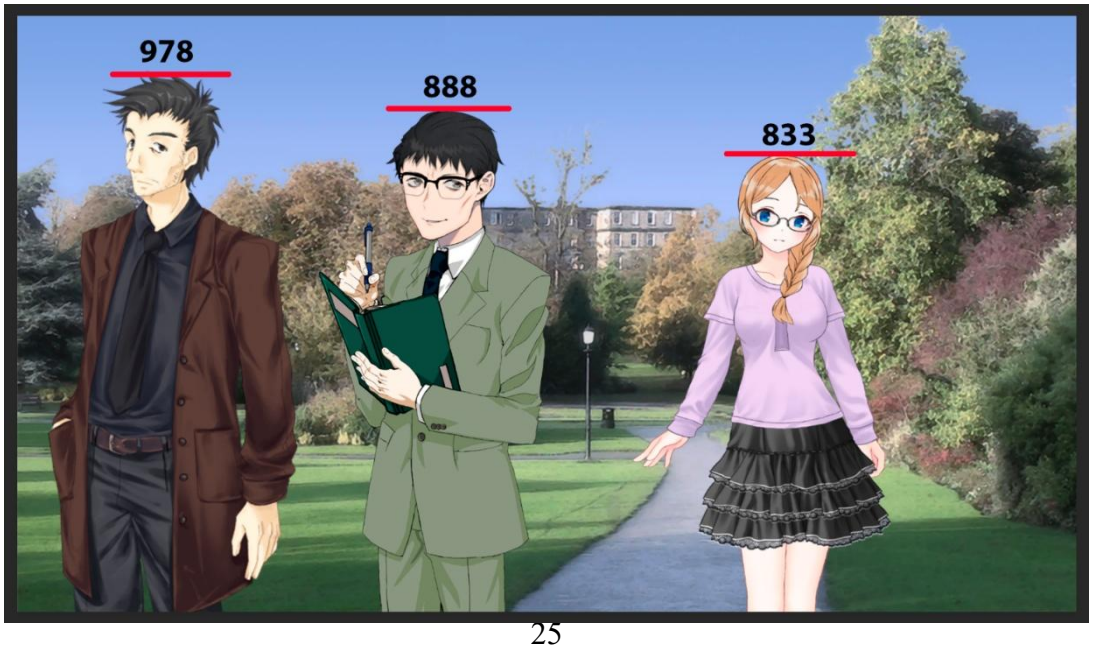
Columna Events

Aquí describiremos todas las acciones relacionadas con los personajes que se verán en pantalla, desde su aparición, destrucción, movimientos, cambios de Sprite y algunas animaciones predefinidas. No obstante, empezaremos definiendo las medidas de los sprites para los personajes.

Todos los sprites deben tener una altura de lienzo de 1080 pixeles, en cuanto al ancho, no hay ningún tipo de límite, solo debes tener en cuenta las siguientes consideraciones. Deben estar en formato .PNG

El personaje será mostrado en pantalla tal cual esté en su Sprite.

Mostrando a personajes con distintas alturas.







Aquí podemos apreciar de izquierda a derecha a Mutou, Tohiro y Keiko. Todos los sprites mostrados tienen un tamaño de lienzo de 1080 pixeles, sin embargo, el personaje en sí tiene una altura distinta. Podemos jugar con la altura para mostrar las diferencias entre un personaje alto y otro bajito.

Habrás notado que el Sprite de Keiko tiene un extraño espacio en blanco en su Sprite, en su lado derecho mientras que los demás se ven más ajustados.

Esto es así para que el Sprite de Keiko quede centrado, si colocamos una línea en toda la mitad del Sprite veremos que el personaje se ve centrado.

El Sprite de Keiko tiene un espacio en blanco



Con ese espacio en blanco, su Sprite se ve centrado.

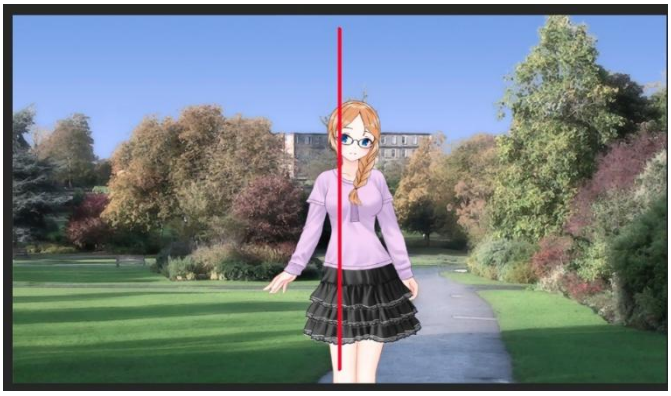


Si recortásemos ese espacio en blanco su Sprite quedaría descentrado de la siguiente manera.

El Sprite de Keiko recortando su espacio en blanco.

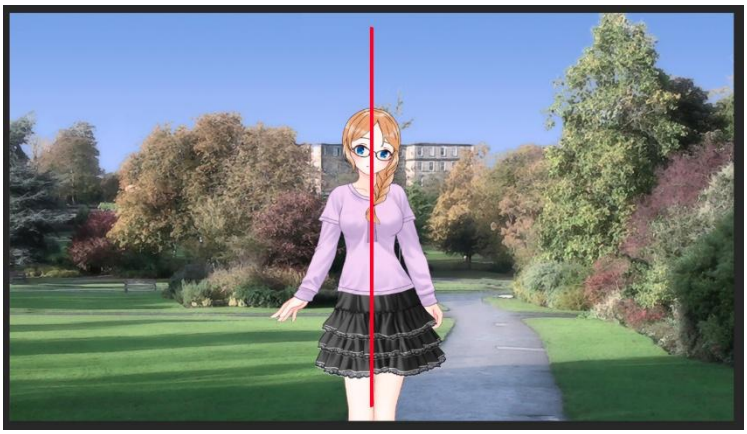


Mostrando a Keiko con el espacio en blanco de su Sprite recortado.



Y así se vería con el Sprite centrado usando espacio en blanco.

Mostrando a Keiko con el espacio en blanco en su Sprite.



En conclusión, aprovecha los espacios en blanco para centrar sprites y evitar que un personaje se mueva bruscamente al hacer un cambio de expresión.

Una vez aclarado las normas para creación de sprites, pasaremos al siguiente apartado que explicará cómo mostrar a nuestros personajes, hacer que se muevan y hagan toda una variedad de animaciones que nos ofrece **Atlax Renpy**.

Primeramente, al momento de colocar a los personajes en pantalla, se debe ser plenamente consciente de lo que está sucediendo en pantalla, es decir, si hacemos que aparezca el personaje “Amber”, ese personaje estará en pantalla hasta que sea destruido.

Mientras esté en pantalla podrá ejecutar acciones como moverse, saltar, temblar, etc. Por lo tanto, si intentas que un personaje que NO esté en pantalla salte o se mueva, el motor arrojará un error porque dicho personaje no está en pantalla.

En cuanto a los distintos de sprites de un mismo personaje, se deben escribir de la siguiente forma, “amber uniform happy left”. Todo el Sprite en minúscula, siempre empezando por el nombre del personaje y seguido del nombre del Sprite específico.

En este caso, para mostrar exitosamente el Sprite “amber uniform happy left” debe existir el archivo “amber uniform happy left.png”.

La recomendación es que al momento de organizar estos archivos lo hagas de la siguiente forma *game/images/characters/[nombre del personaje]/Sprite*.

Algo muy interesante de **Atlax Renpy** es que en ocasiones querremos que los sprites de nuestros personajes se volteen horizontalmente, podrías llegar a pensar en copiar el Sprite “amber uniform happy left”, pegarlo bajo el nombre de “amber uniform happy right” y voltearlo verticalmente con un editor de imágenes, sin embargo, **Atlax Renpy** cuenta con un sistema que ayuda a ahorrar ese tipo de trucos.

Si intentas mostrar “amber uniform happy right”, el motor intentará buscar el Sprite, si no lo encuentra buscará entonces “amber uniform happy left”, si lo encuentra lo volteará automáticamente y viceversa.

Para que el motor use este sistema de manera efectiva, procura que “left” o “right” esté escrito al final del sprite.

Por supuesto, no todos los personajes son simétricos, si un personaje tiene un parche, al voltearlo tendrá el parche del otro lado, en esos casos sí que tendremos la obligación de crear sprites de “left” y “right” manualmente.

A lo largo de esta sección se hará referencia numerosas veces a la “posición”, esta posición consta de un número entero que representa un porcentaje del ancho de la pantalla.

Posiciones y valores

Valor	Posición
-20	El personaje aparece completamente oculto fuera de la pantalla por la izquierda.
0	El personaje se asoma por la izquierda, la mitad de su Sprite es visible.
15	El personaje completamente visible en la parte más a la izquierda de la pantalla.
33	El personaje aparece ubicado a un tercio del ancho de la pantalla.
50	El personaje ubicado en todo el centro.
66	El personaje aparece ubicado a dos tercios del ancho de la pantalla.
85	El personaje completamente visible en la parte más a la derecha de la pantalla.
100	El personaje se asoma por la derecha, la mitad de su Sprite es visible.
120	El personaje aparece completamente oculto fuera de la pantalla por la derecha.

Esos son los valores y medidas de referencia, por supuesto que puedes colocar el número que tú desees al momento de ubicar tus personajes en pantalla. Si tu personaje es muy ancho, es posible que las posiciones -20 y 120 no sean suficientes para ocultarlo del todo, por lo tanto, tendrás que colocar valores más grandes.

Una vez dejado claro estos aspectos técnicos sobre los sprites, su nombre y posiciones, avancemos directamente a los eventos. Estos serán divididos en dos, eventos de aparición y eventos de personajes ya en pantalla.

Comencemos con los eventos de aparición de personajes. Su sintaxis es la siguiente.

Nombre del personaje Sprite : método de aparición : parámetros de la aparición

Importante: Si presentas problemas al momento de mostrar personajes, por ejemplo, que el Sprite del personaje se cambia, verifica que estás utilizando el nombre del personaje con su respectivo sprite, por ejemplo: katherine left happy, berto right surprised, en cada uno de los eventos que escribas.

Eventos de aparición de personajes

Nombre	Descripción	Parámetros
pop	El personaje aparece de golpe.	1. La posición donde aparecerá (50)
appear	El personaje aparece invisible y se va haciendo visible a lo largo de un tiempo dado. (aparición por defecto)	1. La posición donde aparecerá (50)
		2. Los segundos que tarda en aparecer (2)

amber casual normal right (Amber aparece en el centro de la pantalla con una aparición de 2 segundos)

katherine uniform left : pop : 33 (Katherine aparece de golpe en un tercio de la pantalla)

calcercio : pop : 120 (Calcercio aparece fuera de pantalla por la derecha para posteriormente moverlo y hacer que aparezca por la derecha)

Eventos de personajes en pantalla

Nombre	Descripción	Parámetros
(Escribir personaje más su Sprite)	El personaje cambia al Sprite dado.	1. Los segundos que el personaje cambia al nuevo Sprite (0)
move	El personaje se mueve a la posición dada.	1. La nueva posición.
		2. Los segundos que tardará en moverse. (2)
jump	El personaje da un salto y regresa a su posición anterior. El salto dura 0.1 segundos.	1. La intensidad del salto (0.025)
decor	Un decorador. Encima del personaje se muestra una imagen, la cual se moverá junto al personaje.	1. El nombre de la imagen a mostrar.
tremble	El personaje tiembla una cantidad de veces, cada tembleque demora 0.05 segundos.	1. Las veces que el personaje tiembla (10)
hitl / hitr	El personaje rota hacia la izquierda (hirl) o derecha (hitr) simulando que acaba de recibir un golpe.	1. La intensidad del golpe (10). La intensidad se basa en grados de inclinación.
knockl / knockr	El personaje es empujado hacia la izquierda (knockl) o derecha (knockr) y es derribado al suelo fuera de pantalla.	1. La duración del derribo en segundos (0.5)

raisel / raiser	El personaje se levanta del suelo desde la izquierda (raisel) o derecha (raiser).	1. La duración del derribo en segundos (0.5)
zoom	El Sprite del personaje se acerca a la pantalla desde su centro.	1. El factor del zoom (2)
		2. La duración del zoom (2)
movey	El personaje se mueve verticalmente.	1. Número decimal que indica hacia donde se moverá el personaje. 0 = personaje en el centro. 1 = Saldrá de pantalla por abajo. -1 = Saldrá de pantalla por arriba.
		2 Duración del movimiento (0.5)
attack	El personaje se agranda y regresa a su tamaño normal por un breve momento, simulando que está atacando al jugador.	Ninguno.
dodge	El personaje se vuelve pequeño y ligeramente opaco para luego volver a su tamaño y opacidad normal simulando que está esquivando un ataque.	Ninguno.
damage	El personaje se mueve un poco en horizontal y regresa a su posición inicial simulando que ha recibido un golpe.	Ninguno.
destroy	El personaje hace una desaparición y luego es eliminado de pantalla	1. Duración de la desaparición (0)

katherine : zoom (Katherine se acerca a la pantalla doblando su tamaño simulando que está muy cerca, como por ejemplo dando un abrazo)

amber : jump (Amber da un pequeño salto, simulando que ha pasado un susto o que está contenta por algo)

calcercio : move : 47 : 0.5 (Calcercio se mueve a la posición 47 en 0.5 segundos, es decir, con muchísima velocidad)

stefannie uniform angry right : hitr (Stefannie con su Sprite uniform angry right se inclina hacia la derecha simulando un golpe)

Así mismo, se pueden realizar un mismo evento para varios personajes, para ello tenemos que escribir a los personajes separados por “&”.

Utilizar eventos de aparición con & para mostrar varios personajes al mismo en pantalla colocará a los personajes de forma equitativa dependiendo de cuantos sean.

stefannie & calcercio : appear (Stefannie y Calcercio aparecen al mismo tiempo, Stefannie al 33% de la pantalla y Calcercio al 66%)

calcercio happy & stefannie : move : 30: 85 (Calcercio cambia de expresión y se mueve a la posición 30 al mismo tiempo que Stefannie se mueve a la posición 85)

En caso de que quieras encadenar una serie de eventos y se ejecuten uno a uno, sepáralos por comas.

stefannie : move : 30, calcercio : move :120 (Stefannie se mueve a la posición 30 y luego Calcercio se mueve a la posición 120 saliendo de la pantalla por la derecha)

amber happy:decor:stars, amber happy:jump, (Amber se pone contenta y se coloca encima de ella un decorador de estrellas y luego da un saltito para hacer énfasis en lo contenta que está)

Los decoradores deben estar en la carpeta *game/images/displayables/decors*.

También existen animaciones continuas, similares a los efectos continuos estos se ejecutarán en bucle hasta que sean llamados nuevamente. Aquí hay algunas animaciones constantes ya definidas en **Atlax Renpy**.

Si necesitas crear animaciones personalizadas, ve al módulo de personalización más adelante en este documento.

Animaciones constantes

Nombre	Descripción	Parámetros
trembling	El personaje tiembla indefinidamente	Ninguno
jumping	El personaje salta indefinidamente con una intensidad dada.	1. La intensidad de los saltos. (0.025)

Importante: **Renpy** no soporta ciertas animaciones al mismo tiempo, por ejemplo no soporta un cambio de fondo y la aparición de un personaje, si hacemos esto el fondo aparecerá de golpe y el personaje hará la aparición normal. Es cuestión de probar e intentar crear diálogos intermedios para poder recrear estas escenas lo más fluidas e inmersivas posible.

Es entendible que los eventos poseen mucha información y parámetros distintos, es por eso que, a continuación, se presentarán unos cuantos escenarios de ejemplo utilizando estos eventos.

1. Saijo aparece y corre hacia una puerta para intentar abrirla dándole golpes
    a. saijo right : appear (Saijo aparece en mitad de la pantalla)
    b. saijo right : move : 90 (Saijo se mueve hacia la derecha cerca del final)
    c. saijo right : hitr (Saijo se inclina hacia la derecha simulando que está golpeando la puerta)

2. El protagonista entra a una habitación donde está Saijo y Dereck, Dereck sale de la habitación, pero luego regresa y nuevamente se retira
    a. saijo right & dereck left (Saijo y Dereck aparecen en pantalla repartidos equitativamente, los sprites se miran entre ellos)
    b. dereck right : move : 125 : 5 (Dereck rota su sprite y ahora mira hacia la derecha, se mueve hasta salir de la pantalla por el lado derecho a lo largo de 5 segundos, con lentitud)
    c. dereck left : move : 90 : 1 (Dereck reaparece por el lado derecho mirando hacia la izquierda)
    d. dereck right : move : 125 (Dereck vuelve a ir a la derecha hasta salir de la pantalla)
    e. dereck : destroy (Como dereck no aparecerá más en esta escena, lo destruimos para ahorrar recursos)

3. El protagonista le juega una broma a Dereck y este se asusta.
    a. dereck left : pop : 120 (Dereck aparece afuera de la pantalla, por el lado derecho)
    b. dereck left : move : 33 (Dereck se mueve hasta un tercio de la pantalla)
    c. dereck surprised right : jump (Dereck cambia su sprtie a “surprised right e inmediatamente después da un salto)

4. Hay una pelea entre Mutou y Dereck, Mutou lo golpea y logra derribarlo pero Dereck se levanta y derriba a Mutou quedando victorioso
    a. mutou : hitl, dereck : knockl (Mutou golpea hacia la izquierda y Dereck cae hacia la izquierda)
    b. dereck : raisel (Dereck se levanta desde la izquierda)
    c. dereck : hitr, mutou : knockr (Dereck golpea hacia la derecha y Mutou cae hacia la derecha)

Columna Delay

En esta columna podemos manejar tres aspectos que pueden llegar a ser interesantes al momento de mostrar el texto de los diálogos.

1. Especificar un tiempo de espera en segundos antes de que se muestre el texto.
2. Establecer la velocidad en la que se muestra el texto.
3. Que el texto se pase automáticamente al terminar de mostrar el texto.

Cabe aclarar que el tiempo de espera para mostrar el diálogo irá después de los movimientos, animaciones, efectos, sonidos, músicas, etc.

Para tener una idea general de la velocidad del texto se presentará la siguiente lista.

- 20 caracteres por segundo: Texto lento.
- 50 caracteres por segundo: velocidad de texto media
- 100 caracteres por segundo: velocidad de texto alta
- 200 caracteres por segundo: Extremadamente rápido.

Por defecto la velocidad de texto será la que el usuario defina en el menú de preferencias del juego. Sintaxis.

Segundos de espera : Velocidad de texto : pass

5:10 (Se esperan 5 segundos antes de mostrar el texto del diálogo y la velocidad del texto tendrá una velocidad bastante lenta)

0:75 (No se espera ninguna cantidad de tiempo y el personaje habla a una velocidad normal-rápida)

0:200 (Sin tiempo de espera, el texto es mostrado a una velocidad increíble)

0:4 (Sin ningún tipo de espera, el personaje habla a una velocidad muy lenta, dando un aire de suspenso)

Para hacer que el texto se pase automáticamente, debemos escribir “pass” como parámetro en cualquier columna, puedes colocar solo “pass” o combinarlo con los parámetros previamente vistos.

pass (Al terminar de mostrar el diálogo, sea en la velocidad que sea, este pasará automáticamente al siguiente)

0:15:pass (Sin espera, a velocidad de 15 caracteres por segundo, cuando se termine de mostrar se pasa el diálogo)

pass:0:180 (Sin espera, el texto se muestra a una velocidad tan alta que seguramente ni si quiera se pueda leer y una vez termine se pasa el diálogo, generando así un efecto de un personaje que habla extremadamente rápido sin ningún tipo de pausa)

En la configuración de **Renpy** es posible colocar la configuración de velocidad de texto al máximo, ocasionando que el texto se escriba de golpe en la caja de texto. En estos casos, si escribimos un “pass”, el texto será pasado inmediatamente sin dar oportunidad a ver el texto.

Si **Atlax Renpy** detecta este escenario, antes de pasar el diálogo, forzará la velocidad del texto a 180 caracteres por segundo, para dar oportunidad al jugador de experimentar dicho diálogo. De todas formas, la velocidad de texto que definas en la

columna delay, siempre será la prioridad absoluta por encima de la configuración de **Renpy** (para ese diálogo).

Por rápido o lento que avance el texto, el jugador siempre podrá consultarlo en la ventana de historial.

Columna Emisor

Es el texto que aparece en el recuadro arriba de la caja de texto, es decir, la persona o entidad que está hablando. El color de las letras del nombre del emisor dependerá de la definición que le hayamos dado en el archivo de configuración *config.csv*, Así como el nombre del personaje dependiendo del idioma del juego.

Para mostrar el nombre de un personaje, debemos escribir su **nombre clave** definido en el archivo de configuración *config.csv*.

Un escenario muy común es que un personaje hable continuadamente por varios diálogos, en esos casos no será necesario que coloquemos su nombre en cada diálogo, con solo colocarlo una vez, su nombre se mostrará en pantalla hasta que se coloque otro o se quite.

Para quitar el nombre de la caja de texto, de manera similar a la columna music, debemos escribir un "*". De esa forma el nombre será removido de pantalla hasta que se coloque uno nuevo.

Repitiendo los nombres para mostrar el nombre del emisor (incorrecto)

amber
amber
amber
stefannie
amber
stefannie
stefannie
*
*
stefannie

Colocando el nombre las veces necesarias para mostrar el nombre del emisor (correcto)

amber
stefannie
amber
stefannie
*
stefannie

Columnas de Idioma

A partir de esta columna se escribirá el texto de los diálogos que serán mostrado en pantalla en los distintos idiomas declarados en la cabecera del archivo de configuración

config.csv. En caso de necesitar añadir más idiomas es tan sencillo como añadir una columna adicional y escribir en la cabecera el nombre del idioma.

En caso de que agregues un nuevo idioma, no olvides generar la traducción utilizando el launcher de **Renpy**, a la vez que agregar el idioma en la pantalla de configuración de *screens.rpy.*

Límite de caracteres

El límite de palabras que puede contener una caja de texto puede depender de la fuente, su tamaño y el tamaño de la caja de texto. Un límite aproximado es 2539 pixeles. Es decir, si escribimos lo siguiente.

> *No se que hora o que día es hoy, creo que es miércoles en la tarde.No se que hora o que día es hoy, creo que es miércoles en la tarde.No se que hora o que día es hoy, creo que es miércoles en la tarde.No se que hora o que día es hoy, creo que es miércoles en la tarde.No se que hora o que día es hoy, creo que es miércoles en la*

Llenaremos por completo la caja de texto con un total de 328 caracteres, pero si escribimos 328 veces "i" veremos que aún queda mucho espacio por rellenar porque el carácter "i" ocupa menos espacio en pixeles.

Para saber cuántos pixeles tiene tu diálogo puedes utilizar el siguiente sitio web: https://webutility.io/pixel-width-calculator siempre es importante saber con precisión el límite de caracteres de tu novela y tenerlo presente.

Modificadores de texto

Renpy posee diversos modificadores de texto, estos se escriben en medio del diálogo y son perfectamente utilizables en las columnas de idioma. Existen muchos modificadores de texto así que te recomiendo leer la documentación de **Renpy** para intentar implementarlos. https://www.renpy.org/doc/html/text.html#text-tag-alpha Si se te hace muy complicado puedes pedirle ayuda al desarrollador de tu proyecto.

Dependiendo de la fuente que utilices en tu proyecto, es posible que ciertos caracteres no se muestren, por ejemplo, el carácter "♪" no existe en la fuente "playtime" y en su lugar se mostrará un carácter extraño. En esos casos tendrás que usar obligatoriamente un modificador de texto especificando una fuente la cual sí que posea el carácter necesitado.

Mostrando el diálogo "Ahora a bailar wuuuu ♪♪♪♪"



El modificador de texto es el siguiente.

{font=DejaVuSans.ttf}♪♪♪{/font}

De esa forma las notas musicales se muestran en la fuente “DejaVuSans.ttf”

Mostrando el diálogo “Ahora a bailar wuuuu {font=DejaVuSans.ttf}♪♪♪{/font}”



Caracteres especiales

Dada la naturaleza del motor y por el lenguaje en el que está escrito, existen ciertos caracteres especiales que el motor no soporta. Por lo tanto, para mostrar estos caracteres especiales, es necesario colocar una serie de caracteres específicos, de esta forma, el motor los reemplazará.

- Escribe [.,] para que el motor muestre [;]

Orden de ejecución

Atlax Renpy tiene el siguiente orden de ejecución al momento de mostrar un diálogo de un archivo de control.

1. Finalización/Inicio de escena.
2. Cambios de fondo
3. Eventos
4. Audios
5. Efectos
6. Efectos continuos y parpadeos
7. Delay (tiempos de espera)
8. Mostrar texto del diálogo.

Finalización de escena

Una vez hayamos terminado la escena, es decir, el archivo de control, hay que darle un final, este final puede ser lineal, una decisión, una consecuencia o un script personalizado.

Una finalización de escena describe la manera en que la escena va a terminar. Para definir el final de una escena debemos ir a la última fila de nuestro **archivo de control** y empezar la columna **Key** con un “#” y acto seguido describir el tipo de final que tendrá nuestra escena con sus respectivos parámetros.

Tipo de finalización de escena ; transición : parámetros de transición
Key_1 ; valor 1 ; valor 2 ; valor 3
Key_ 2 ; valor 1 ; valor 2 ; valor 3

Las siguientes filas luego de definir el tipo de finalización, son cada uno de los elementos que acompañan al tipo de finalización, si se tratase de una decisión, estos elementos serían las opciones posibles a escoger.

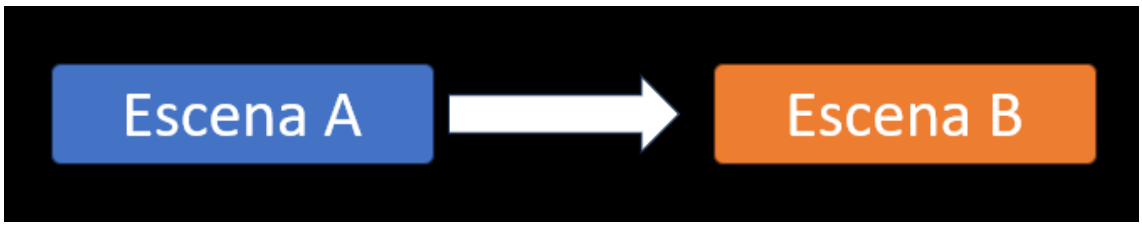
En este caso nombrados “Key_1” y “Key_2”. Es similar a la columna **Key** de los archivos de control, es decir, tienen que tener un nombre único, exclusivo e irrepetible en toda la novela visual, ya que estos **Key** serán los que usará el motor para definir consecuencias.

Existen varios tipos de finalización de escenas, es importante que las conozcas todas para poder sacar el máximo provecho a tu historia. Si tu historia no tiene decisiones ni ramificaciones, céntrate en el tipo de finalización lineal, los demás no serán necesarios.

Finalización lineal

Una escena que termina de forma lineal es la que siempre lleva a la misma escena. Útil cuando queremos que varios caminos desemboquen en uno solo o simplemente para mantener más ordenados los archivos de control.

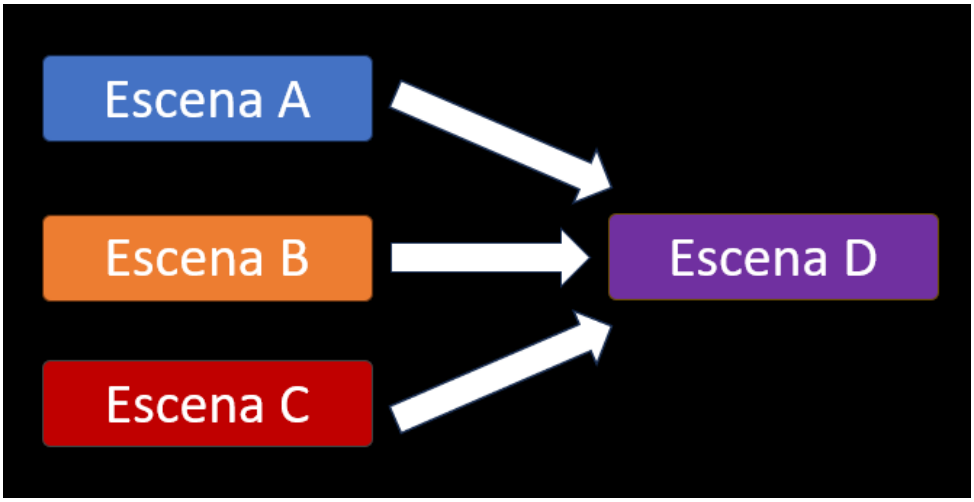
La escena A siempre llevará a la escena B



En este ejemplo podemos ver como la Escena A siempre conlleva a la Escena B. Si bien es cierto que podríamos obviar este salto y escribir ambas escenas en un mismo archivo, lo recomendable es que cada escena englobe una situación para así facilitar su organización.

Si escribiéramos todo el capítulo en un solo archivo tendríamos un archivo de control demasiado extenso, ilegible y difícil de rastrear.

Las escenas A, B y C siempre conducen a la escena D



Aquí podemos apreciar otro caso de finalización lineal, Las escenas A, B y C siempre terminan en la escena D. Es decir, el autor desea que la escena D siempre suceda sin importar el camino que hay tomado el jugador.

Un escenario bastante usual es cuando ocurre una decisión y al final los caminos retoman en una misma escena.

La sintaxis de una escena que termina linealmente es la siguiente.

#linear
Key_de_fin_de_escena_A ; Nombre_de_la_siguiente_escena

Por ejemplo, si estamos en una escena llamada "studying_for_final_exam" y tenemos otra escena que le sigue llamada "the_final_exam", el archivo de control studying_for_final_exam.csv debería de terminar de la siguiente forma.

#linear
studying_for_final_exam_end ; the_final_exam

Continuando con el ejemplo de la segunda imagen, imaginemos que en nuestra historia hay un examen obligatorio que sin importar la ruta en que se encuentre el jugador, este examen siempre va a suceder. La escena del examen la llamaremos "the_x_exam".

(En la ruta de Katherine)

#linear
katherine_pre_x_exam ; the_x_exam

(En la ruta de Mariangeles)

#linear
mariangeles_pre_x_exam ; the_x_exam

(En la ruta de Amber)

#linear
amber_pre_x_exam ; the_x_exam

(En la ruta de Stefannie)

#linear
stefannie_pre_x_exam ; the_x_ex

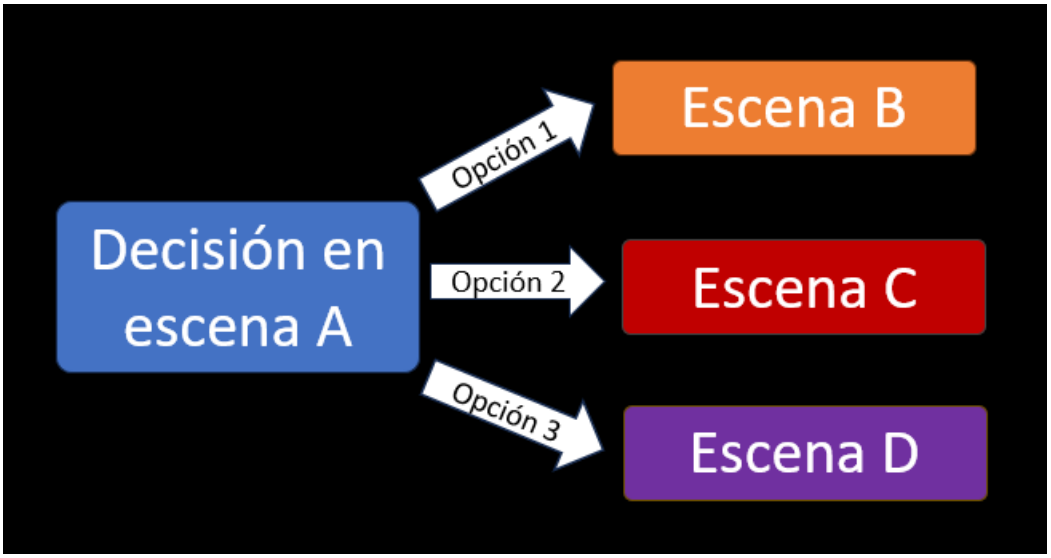
Finalización con decisión

La decisión es la mecánica principal de diversas novelas visuales, este consiste en plasmar frente el jugador una situación en la que deba escoger una opción de entre varias mostradas en pantalla. Con las decisiones podemos dividir la historia en caminos distintos y además **Atlax Renpy** nos permitirá agregar un sistema de puntos. Sintaxis.

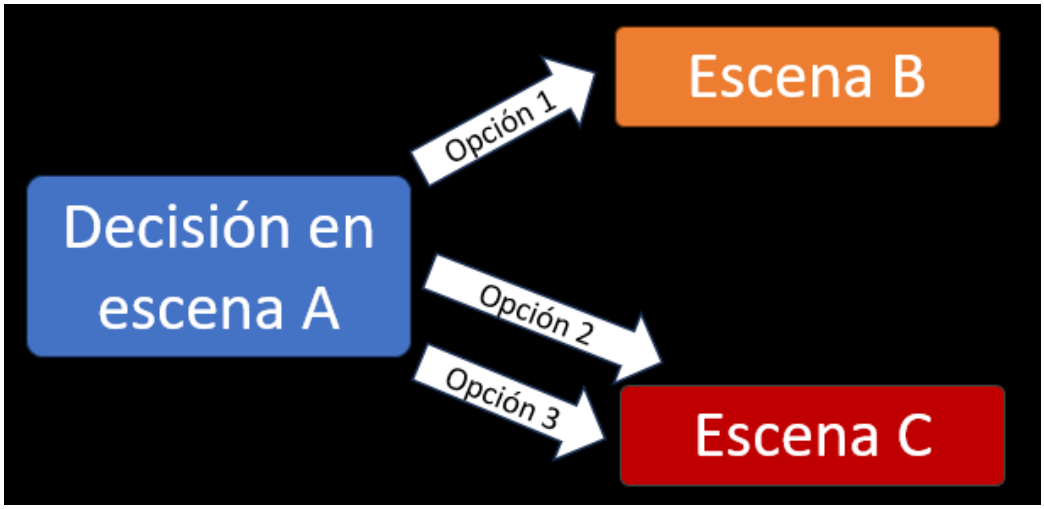
```
#decision
key_opcion_1 ; punto_1 : valor_punto_1 ; siguiente escena ; texto
key_opcion_2 ; ; siguiente escena ; texto
key_opcion_3 ; punto1:valor_punto1, punto2:valor_punto2 ; siguiente escena ; texto
```

Podemos hacer que cada opción de la decisión lleve a una escena distinta o que algunas de ellas lleven a la misma.

La decisión A tiene tres opciones que llevan a 3 escenas distintas.



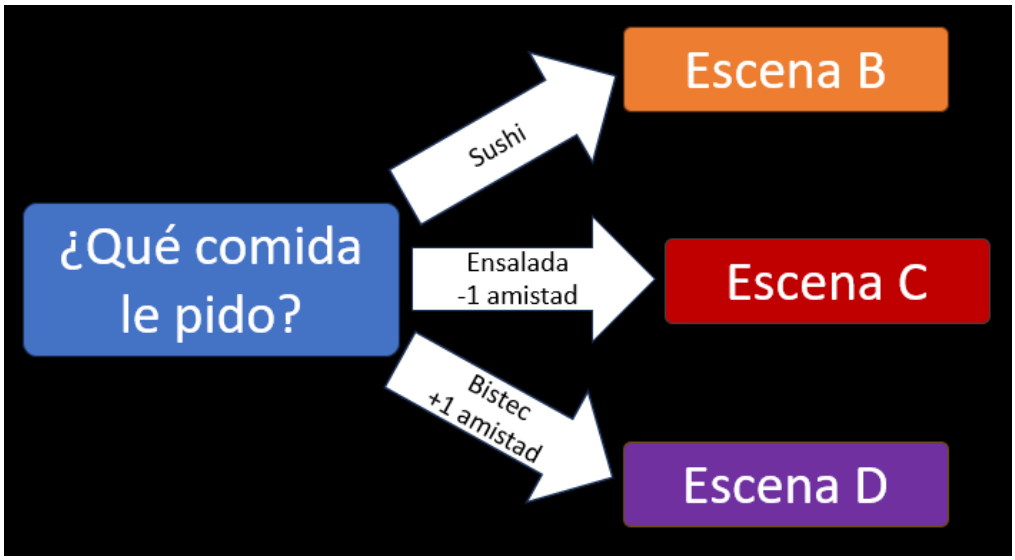
La decisión A tiene tres opciones y dos de ellas llevan a la misma escena.



También se pueden modificar las **puntuaciones**, estos puntos pueden llevar el nombre que desees. Asegúrate de escribir estos puntos exactamente con el mismo nombre, respetando mayúsculas y minúsculas ya que si te equivocas, el motor lo interpretará como una **puntuación** distinta.

A continuación, se mostrará una decisión que modifica la **puntuación** de amistad.

Dependiendo de la comida que se escoja, ganarás o perderás puntos de amistad.



Por supuesto que los puntos no son obligatorios, puedes dejarlo en blanco. El ejemplo anterior se escribiría así.

#decision
give_sushi ; ; scene_B ; Sushi (No afecta a la puntuación)
give_salad ; amistad : -1 ; scene_C ; Ensalada ; Salad (Resta 1 de amistad)
give_steak ; amistad : 1 ; scene_D ; Bistec ; Steak (Suma 1 de amistad)

Al igual que con los diálogos, el texto de la decisión debe escribirse respetando el orden de los idiomas especificados en *config.csv*. Si una opción carece de un texto en el idioma del juego, mostrará la primera por defecto.

Nuevamente hago énfasis en que cada opción debe tener su identificador único ya que nos servirá para manejar las **consecuencias**, en este caso son "give_sushi", "give_salad" y "give_steak", estos identificadores jamás deben repetirse, o al menos, en la misma **ruta**.

Finalización por condición

Cuando la escena llega al final, dependiendo de una serie de condiciones, el motor automáticamente toma un camino sin que el jugador lo note. Esto nos permite hacer que una escena conlleve a múltiples escenas sin necesidad de colocar una decisión, en vez de eso, nos basaremos en eventos pasados como si se tratara de una **consecuencia**.

Existen 3 tipos de condiciones, condición por puntos, por escena y por decisión. Para estas condiciones se debe tener especial control sobre los valores posibles de los puntos, así como los posibles diálogos y escenas pasadas, esto para saber si de verdad las condiciones que estás escribiendo se pueden cumplir.

Condición por puntos

Una condición por puntos es aquella que compara una **puntuación** específica del jugador con un valor preestablecido, si la condición se cumple, el juego tomará un camino y sino, tomará otro.

Las condiciones por puntos son excelentes herramientas para definir finales buenos y malos. Sin embargo, debes estudiar primero las posibles puntuaciones que podría tener el jugador en ese punto, ya que, si no lo piensas bien, podrías crear condiciones imposibles o muy difíciles de cumplir.

Por ejemplo, si colocas la condición "amistad mayor o igual a 50" y que a esas alturas, el jugador como mucho pudiese tener 10 de amistad, estarías creando una condición imposible, por lo tanto, el juego siempre tomaría el otro camino.

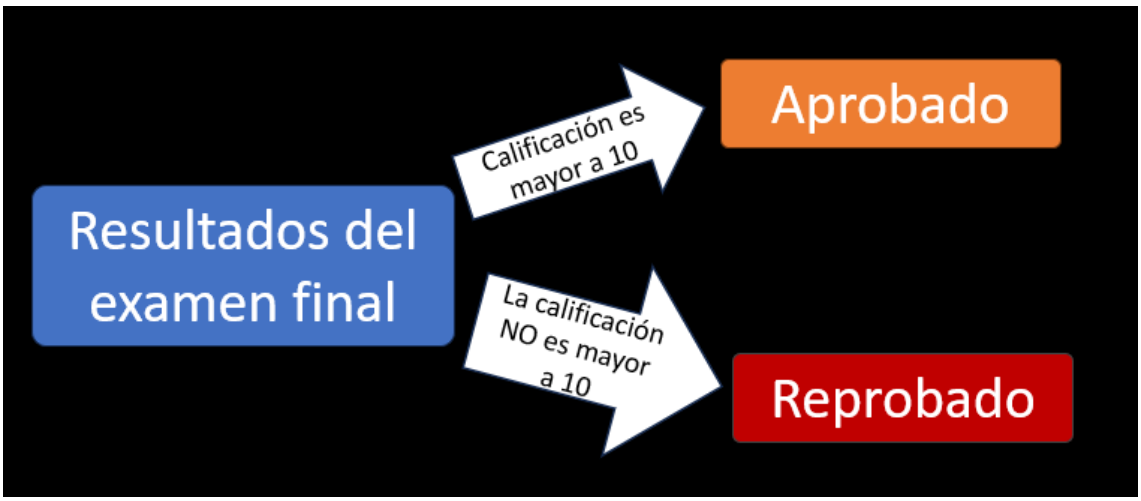
En ese caso tendrías que ajustar la condición y/o las decisiones anteriores que añadan puntos de amistad.

Más adelante en la sección **balanceo de novela visual**, presentaré una herramienta llamada **VNTA** para realizar estos ajustes.

Los distintos operadores de comparación son:

1. < (menor)
2. > (mayor)
3. <= (menor o igual)
4. >= (mayor o igual)

Dependiendo de la calificación, el jugador aprobará el examen o no



#condition points
final_exam_passed ; calificación > 10 ; aprobado (Si tiene más de 10 de calificación mandas a la escena *aprobado.csv)*
final_exam_failed; ; reprobado (Si no, entonces manda a la escena *reprobado.csv)*

Si **Atlax Renpy** verifica todas las condiciones y ninguna se cumple toma la última por defecto. No hace falta colocar condición en la última opción.

También se pueden colocar condiciones múltiples, en ese caso lo separaremos por comas.

#condition points
katherine_route_good_end ; amistad > 18, calificación > 10 ; final_bueno
katherine_route_neutral_end ; amistad > 12, calificación > 5 ; final_neutral
katherine_route_bad_end ; ; final_malo

Si el jugador alcanzó más de 18 puntos de amistad y 10 puntos de calificación, obtendrá un final bueno.

Si no, si alcanzó más de 12 puntos de amistad y 5 de calificación, entonces obtendrá el final neutral.

Si ninguna de las dos se cumple, entonces irá al final malo.

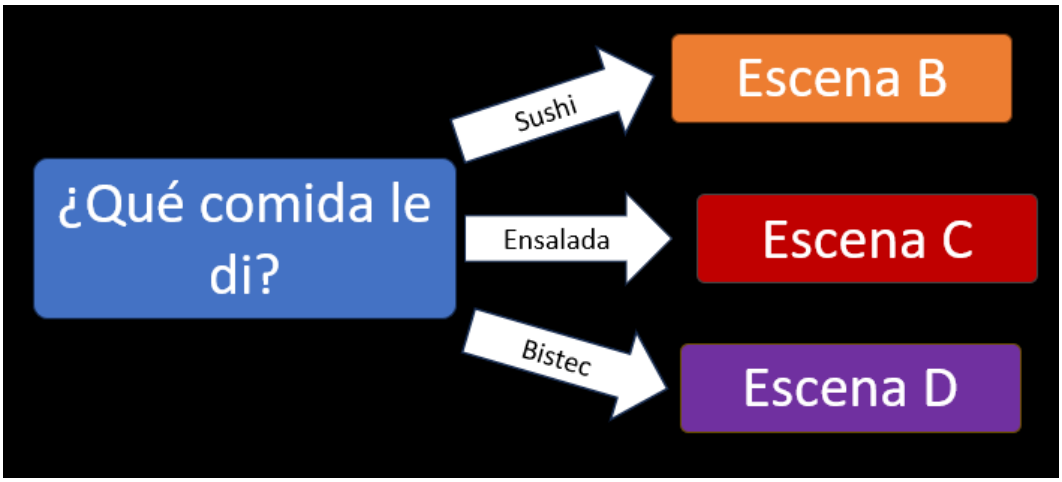
Condición por decisión

La condición por decisión es probablemente el tipo de finalización más común, esta condición se basará en las decisiones pasadas tomadas por el jugador.

¿Recuerdan los identificadores que con tanta insistencia se dijo que los hicieran únicos? Para esto era, para poder crear caminos a partir de esas decisiones pasadas, se necesitará el Key de la opción a la que queramos hacer referencia, esto será más claro con un ejemplo.

Usando como ejemplo la decisión anterior, la de "¿Qué comida le pido?" vamos a crear una condición por decisión.

Condición por decisión basada en la decisión anterior de "¿Qué comida le pido?"



#condition decisión

sushi_gived ; give_sushi ; escena_B (Si se escogió la opción give_sushi manda a la escena B)

salad_gived ; give_salad ; escena_C (Si se escogió la opción give_salad manda a la escena C)

beef_gived ; give_beef ; escena_D (Si se escogió la opción give_beef manda al a escena D)

De forma similar a los puntos, también podemos colocar varios Key separados por comas, en ese caso, el motor requerirá que el jugador haya tomado todas las decisiones allí descritas.

Condición por escena

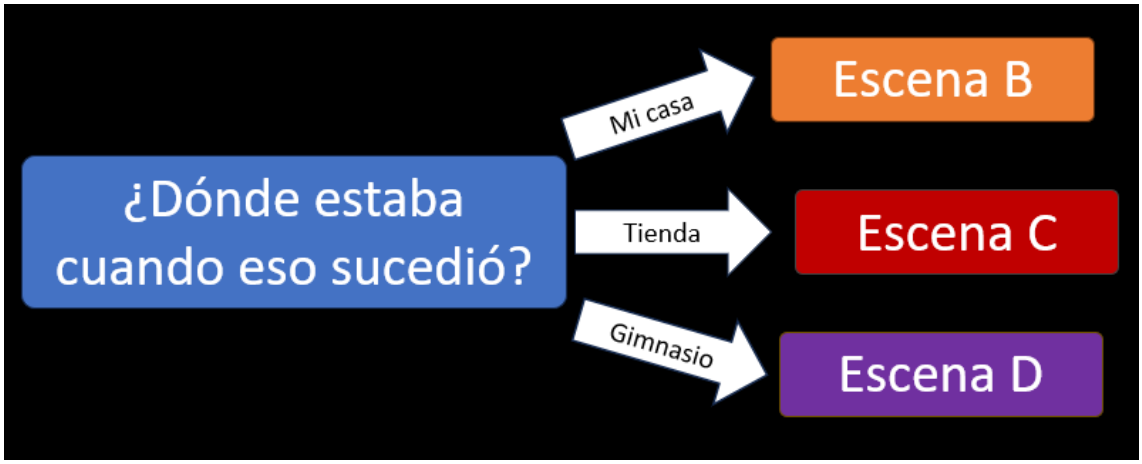
Podemos bifurcar escenas en función a las escenas vistas anteriormente, estas escenas tomarán el nombre del archivo de control correspondiente para hacer la comparación.

Imaginemos que en nuestra historia hay un terremoto que siempre ocurre, solo que, dependiendo de las decisiones del jugador, este puede que esté en varios lugares cuando el terremoto suceda, viviendo así experiencias distintas.

Digamos que el jugador puede vivir el terremoto en tres sitios distintos, en su hogar, en la tienda o en el gimnasio, estas escenas se llaman *earthquake_in_home*, *earthquake_in_shop* y *earthquake_in_gym*.

En un diálogo posterior el protagonista cuenta lo que vivió en el terremoto y dependiendo del lugar en el que estuvo, lo cuenta de una manera u otra.

Dependiendo de una escena jugada, encaminamos al jugador a otra escena.



#condition scene
earthquake_lived_in_home ; earthquake_in_home ; tell_home_earthquake
earthquake_lived_in_shop ; earthquake_in_shop ; tell_shop_earthquake
earthquake_lived_in_gym ; earthquake_in_gym ; tell_gym_earthquake

Quizás sea algo confuso de entender, solo ten en cuenta que la primera columna es el identificador único de la opción que estamos escribiendo, la segunda es el nombre del archivo de control que el jugador debió haber jugado y la tercera columna es el nombre del siguiente archivo de control.

De esa forma, el juego verifica ¿El jugador pasó por la escena "earthquake_in_home.csv"? Entonces lo mandamos a la escena tell_home_earthquake. Y así sucesivamente con las demás opciones. Si ninguna se cumple se tomará la última.

Scripts personalizados

Si bien **Atlax Renpy** es una potente herramienta, tiene una gran limitante, y es que solo puede trabajar con escenas, diálogos, personajes y efectos, si quisieras implementar un minijuego en tu novela, sería totalmente imposible utilizando **Atlax Renpy**.

No obstante, **Atlax Renpy** cuenta con la función de poder ejecutar **scripts personalizados** escritos en lenguaje **Renpy**.

Evidentemente para utilizar estos **scripts personalizados** requerirás de conocimientos en **Renpy**. La manera en que ejecutas **un script personalizado** es al final de una escena, cuando el script personalizado finalice, este deberá de arrojar una respuesta, dicha respuesta es recibida por el motor y enviará al jugador a otra escena dependiendo de la respuesta recibida.

La manera en que se debe definir la respuesta de un **script personalizado** es escribiendo es lo siguiente dentro del script:

$ dialogueManager.customScriptResponse = 'mi respuesta'

Sin importar lo que hagamos en nuestro script, siempre debemos retornar una respuesta, así el motor sabrá a dónde encaminar al jugador.

#script ; label del script personalizado
Key_1 ; expected response 1 ; next_scene_1
Key_2 ; expected response 2 ; next_scene2
Key_3 ; expected response 3 ; next_scene_3

La primera columna corresponde al identificador único del resultado obtenido. La segunda columna es el valor esperado, y la tercera columna es la siguiente escena que será ejecutada si se recibe dicho valor esperado.

Por ejemplo, si creamos un **script personalizado** en el que se juega al ajedrez, dependiendo de quién gane (o si empatan) tendremos que darle un valor a la variable dialogueManager.customScriptResponse.

Suponiendo que hay 3 opciones posibles, ganar, perder y empatar, la finalización de la escena anterior a la del ajedrez, sería parecida a la siguiente.

script ; playing_chess (Esto ejecuta el script personalizado *playing_chess.rpy*)
chess_win_agaisnt_amber ; win ; won_chess_agaisnt_amber
chess_lose_agaisnt_amber ; lose ; lose_chess_agaisnt_amber
chess_draw_agaisnt_amber ; draw ; draw_chess_agaisnt_amber

Luego, dentro del script cuando se defina si el jugador ganó, perdió o empató, deberás asignarle el valor correspondiente al customScriptResponse, algo así.

$ dialogueManager.customScriptResponse = 'win' (Ganó)
$ dialogueManager.customScriptResponse = 'lose' (Perdió)
$ dialogueManager.customScriptResponse = 'draw' (Empató)

Una vez termine el **script personalizado**, el motor leerá la respuesta y dependiendo de esta, enviará al jugador al archivo de control correspondiente (tercera columna).

Consideraciones

Los **scripts personalizados** son una herramienta extremadamente versátil que nos permite implementar cualquier escenario programándolo en lenguaje **Renpy**, pero es necesario dejar claro ciertas consideraciones al momento de crear estos scripts, ya que pueden generar errores o comportamientos extraños.

- Los personajes en pantalla antes de ejecutar el script personalizado no deben ser movidos ni destruidos.
- Si en el script personalizado aparecen nuevos personajes, esos personajes al final deben ser destruidos antes de retomar el flujo normal.
- Si no hay personajes en pantalla antes de ejecutar el script personalizado, al terminar, tampoco debe haber personajes en pantalla.

- En el archivo *config.csv* puedes escribir el label de un script personalizado en el campo “begin”, en estos casos obligatoriamente tienes que escribir lo siguiente al final del script personalizado.
    - $ dialogueManager.NovelBeginWithCustomScript(‘Nombre del siguiente archivo de control’)
    - return
- Si se desea que el final de la novela visual se realice con un script personalizado, al final de esta debemos colocar la siguiente línea de código para indicar que termina el juego. Lo que sucederá después es que se moverá a la pantalla del menú de inicio con un fade.
    - $ dialogueManager.endGame = True
- Los scripts personalizados pueden estar en cualquier lugar del proyecto, sin embargo, se recomienda ubicarlos en la carpeta *game/custom_scripts*.
- Al mostrar un personaje en pantalla obligatoriamente debes colocar “onlayer characters”, de lo contrario el personaje se mostrará en una capa distinta y no será visible.

Siempre que cumplas con las consideraciones, podrás ejecutar estos scripts con total normalidad y así crear escenas imposibles utilizando **Atlax Renpy**.

Finalización mostrando los créditos

Cuando nuestra novela termine, generalmente querremos mostrar los créditos, dentro de la carpeta *game/custom_scrtips* encontrarás un archivo llamado *my_credits.rpy*, ese archivo tendrá un label llamado my_credits con ciertos estilos y facilidades para crear tus propios créditos. Evidentemente estos créditos se hacen en lenguaje **Renpy** así que si quieres cambiarlos, tendrás que tener ciertos conocimientos.

En el momento que nuestra novela termine y deseemos mostrar los créditos, en el método de finalización debemos colocar.

```
# credits
```

Al hacer eso **Atlax Renpy** intentará ejecutar el label “my_credits” y luego mandará al jugador a la pantalla del menú principal con un fade.

Finalización a la pantalla de título

Es posible que solo queramos mostrar los créditos de nuestra novela únicamente en ciertos finales, para el resto podremos optar por enviar al jugador a la pantalla de título. Para hacer esto simplemente escribiremos el siguiente tipo de finalización de diálogo.

```
# title
```

Al escribir #title como método de finalización, el juego hará un Fade extenso al fondo de la pantalla de título y terminará enviando al jugador a la pantalla del título con una transición suave. Algo interesante que podrías hacer es utilizar este método para los

finales malos y neutrales, mientras que, por otro lado, muestras lo créditos en los finales buenos.

Reproducir video tras terminar una escena

Es posible que al pasar de una escena a otra queramos reproducir un video, ya sea una pelea, una presentación del escenario o una animación. Esto lo podemos definir en la misma línea donde definimos el tipo de finalización de diálogo. Cabe destacar que, en **condiciones** y **decisiones**, el video se reproducirá siempre sin importar la opción o camino que se tome. Sintaxis.

#finalización ; video : nombre del video : siguiente fondo

El video debe estar en formato .webm y estar guardado en la carpeta *game/videos.* El motor hará un Dissolve a una pantalla en negro antes de mostrar el video.

Cuando se reproduzca un video el motor hará lo siguiente:

1. Hará un fundido a negro tapando toda la pantalla.
2. Si se especificó un siguiente fondo, ese fondo será colocado.
3. El video se reproduce
4. Se retoma el fundido a negro y se muestra el fondo.

Si no se especifica ningún siguiente fondo, al terminar de reproducirse el video, mostrará el fondo que ya se estaba colocado antes.

#linear ; video : watching the library (Muestra el video watching the library sin cambiar el fondo)

#linear ; video : running to home : home door (Muestra el video running to home y al finalizar hará un fundido al fondo home door)

Limpieza del juego

Por razones de ahorro de recursos es conveniente que de vez en cuando limpiemos nuestra escena, esto lo hacemos añadiendo la opción “clear” como parámetro al final de un fin de diálogo, este debe ir separado por un ;

#decision ; video ; sun_rising ; clear (Reproduce el video sun_rising y luego limpia el juego)
#condition decisión ; ; clear (Limpiando sin hacer una transición)

Siempre que se vaya a pasar de una escena a otra de forma limpia, es decir, sin personajes en pantalla, es conveniente siempre agregar el parámetro clear para ahorrar recursos al dispositivo.

Ejemplos

Junto con este archivo se encuentran dos carpetas llamadas “*ejemplo1*” y “*ejemplo2*”, ubicadas en *game/*scenes. Estas contienen unos archivos de control que explican dos escenas distintas utilizando las funciones que anteriormente se han descrito

a lo largo del documento. Siéntete libre de probarlas, leerlas, y usarlas como referencia al momento de escribir tu novela y/o hacer pruebas.

Cabe aclarar que los recursos visuales utilizados para crear estos ejemplos fueron generados con inteligencia artificial. Esto se hizo así ya que, estos ejemplos tienen fines netamente educativos para demostrar cómo funciona el motor. Siempre procura crear los recursos visuales de tus proyectos sin inteligencia artificial (o al menos no en su totalidad).

De igual manera las canciones de este ejemplo también han sido generadas con inteligencia artificial, sin embargo, estas tienen una mala calidad debido a que obligatoriamente estas canciones requerían ser comprimidas para poder acoplarlas al instalador del motor y ser subido a **Github**. Puedes encontrar las canciones originales en el repositorio oficial de **Github**.

Encriptar archivos de control

Los archivos de control son archivos en formato .CSV separados por comas, es decir, cualquier usuario final va a tener la capacidad de abrir los archivos de control y modificar su interior.

Es entendible que, como creador, desees proteger tu obra, así que **Atlax Renpy** viene con una aplicación que te ayudará a encriptar tus archivos de control. Esto no los hará imposible de descifrar, pero quedarán mucho más protegidas.

La configuración de esta encriptación la podremos encontrar en el archivo *game/definitions/AtlaxEncryptionKey.rpy.* Este archivo solo contiene 2 líneas de código definiendo 2 variables, una llamada requireSceneEncryption y otra atlax_encryption_key. Por defecto el motor trabajará sin ningún tipo de encriptación.

Contenido del archivo AtlaxEncryptionKey.rpy

```
define requireSceneEncryption = False
define atlax_encryption_key = '344ZJLj-TfKZQ189ez5-o-QQC4DkPMnPghG2PJCi3XE='
```

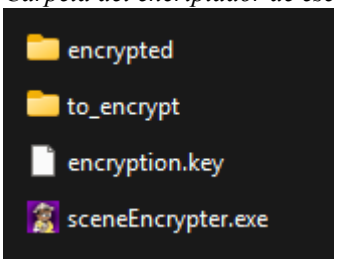
Si requireSceneEncryption dice True, significa que el motor espera que los archivos de control estén encriptados, por lo tanto, los desencriptará utilizando el atlax_encryption_key.

Si requireSceneEncryption dice False, el motor asumirá que los archivos no están encriptados y los leerá tal cual.

La atlax_encryption_key es una cadena de caracteres que funciona como la llave que utiliza el motor para desencriptar los archivos, la llave mostrada anteriormente es un simple ejemplo, puedes utilizar la cadena de caracteres que mejor te parezca y guardarla en algún lado de tu PC o en un papel.

Si deseas encriptar tus archivos de control deberás dirigirte a la carpeta *apps/sceneEncrypter*. Allí encontrarás los siguientes archivos.

Carpeta del encriptador de escenas (es posible que algunos íconos cambien)



Un ejecutable, dos carpetas y una llave de encriptación. La llave de encriptación (encryption.key) puedes abrirlo con un simple editor de texto como el bloc de y esta contiene la llave de ejemplo anteriormente mencionada (recuerda que puedes cambiarla).

Al abrir el sceneEncrypter.exe este tomará todo el contenido de la carpeta *to_encrypt* y lo encriptará volcando su contenido en la carpeta *encrypted*. Cabe destacar que el contenido de *to_encrypt* sólo será leído.

IMPORTANTE: Dada la naturaleza de este ejecutable, es perfectamente posible que el antivirus de tu dispositivo lo detecte como una amenaza y tendrá toda la razón, ya que el comportamiento de este ejecutable es encriptar una serie de archivos, es decir, el mismo funcionamiento que tendría un virus del tipo **Ransomware**.

Asegúrate que utilizas la misma llave escrita en *encryption.key* y en *AtlaxEncryptionKey.rpy.*

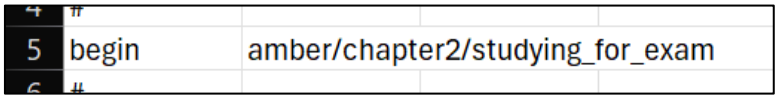
La idea es colocar en la carpeta *to_encrypt* todos los archivos de control con su respectivo sistema de carpetas, ejecutar el *sceneEncrypter.exe* y listo, en la carpeta *encrypted* tendrás los archivos de control encriptados con la llave ubicada en *encryption.key.* Esto debes hacerlo de último, es decir, cuando tu proyecto esté finalizado y listo para ser publicado.

Recuerda que la encriptación es totalmente opcional, es simplemente una capa de seguridad para evitar que los usuarios finales puedan cambiar tu videojuego y redistribuirlo con sus modificaciones. Por defecto esta estará desactivada.

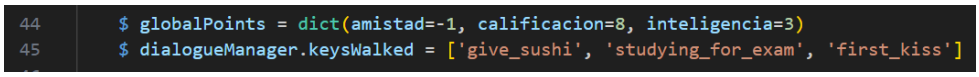
Testing

A medida que escribamos nuestra novela, notaremos que los archivos de control se irán acumulando y quizás algunos de estos alcanzan unos tamaños considerables. Cuanto más grande sea el proyecto, más importante será probarlo y testearlo para asegurarse de que todo funciona de la manera esperada.

Esta sección contendrá una serie de facilidades, consejos y buenas prácticas para poder probar nuestra novela visual.

1. Si deseas probar una escena en específica utiliza el archivo *config.csv* y cambiar el valor de la variable “begin” por la ruta y nombre del archivo de control. Por ejemplo, si está ubicado en *amber/chapter2/studying_for_exam.csv,* en el archivo de configuración colocarías lo siguiente.
    a. 

2. Se puede llegar a ser tedioso tener que empezar la escena desde el inicio para probar que una parte cerca del final funcione correctamente, para ello podemos escribir en la columna Key del archivo de control un ”*”. Al colocar un asterisco, el motor saltará todos los diálogos hasta llegar al diálogo marcado, desde allí empezará a ejecutar esa escena.
    a. Ten en cuenta que la escena al iniciar en ese punto, ignorará por completo los fondos, personajes y músicas previas.

3. Para probar consecuencias de tipo puntos, escenas o de decisión, en el archivo *games/script.rpy* puedes modificar las variables *globalPoints* y *dialogueManager.keysWalked* ubicadas aproximadamente en la línea 44 y 45. Al modificarlo, podrás establecer de manera artificial los puntos y decisiones y/o escenas que necesites. Aquí un ejemplo.
    a. 
```
44    $ globalPoints = dict(amistad=-1, calificacion=8, inteligencia=3)
45    $ dialogueManager.keysWalked = ['give_sushi', 'studying_for_exam', 'first_kiss']
```
    b. La variable keysWalked guarda los keys de las decisiones, nombres de escenas, finalizaciones de diálogo y scripts personalizados.
4. Si al pasar de una escena a otra deseamos que el fondo cambie con una transición específica, tendremos que colocar en el inicio de la nueva escena un cambio de fondo con la transición.

Personalización

La personalización de **Atlax Renpy** es la misma que podrías hacer utilizando **Renpy** puro, lo único personalizable son algunos aspectos que requieren de ciertos conocimientos de programación, los archivos destinados a personalización son *game/definitions/customConfig.rpy p*ara definir configuraciones específicas de **Renpy** y el archivo *game/animations/animations.rpy* para crear animaciones personalizadas en los personajes.

Animaciones personalizadas

Para implementar animaciones personalizadas, primero debes definirlas en *animations.rpy* utilizando lenguaje **Renpy**, específicamente definiendo los tiempos y las propiedades transform. Como ejemplo vamos a crear una animación llamada “dance”, esta hará que el personaje se balancee de un lado a otro.

Primero debemos definir la animación en *animations.rpy*.

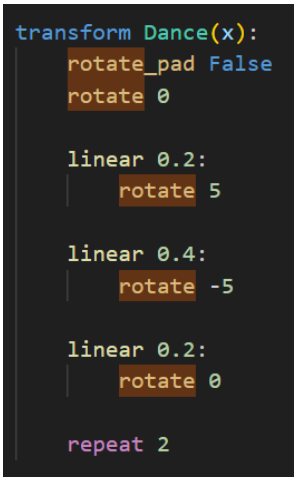
Definiendo la animación Dance



Al momento de crear una animación personalizada, de manera obligatoria debemos hacer que el transform reciba un parámetro x. Dicho parámetro corresponde a la posición del eje X del personaje. Aunque nuestra animación no interactúe con la posición X del personaje, debemos colocar el parámetro de todas formas ya que **Atlax Renpy** intentará pasárselo como parámetro.

Ya definida la animación, debemos dirigirnos al archivo *game/managers/EventManager.rpy*, allí encontraremos una variable llamada *characterAnimations*, la cual contendrá una lista con los nombres de las animaciones permitidas. Agregaremos "dance".

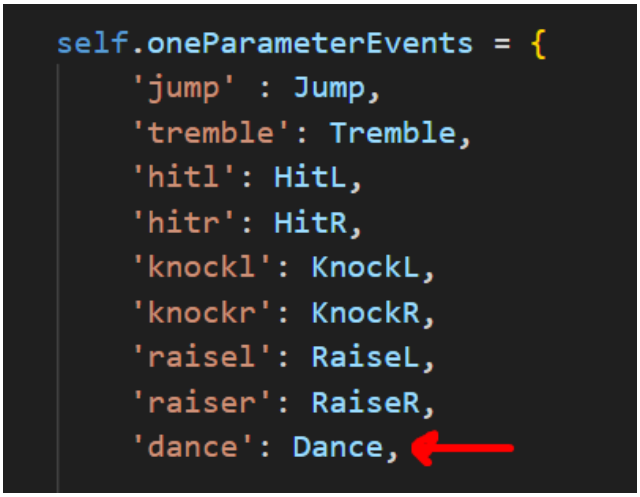
Agregando "dance" a la lista de nombres de animaciones



Recordemos también agregar al final del nombre una coma.

Por último, en el mismo archivo *EventManager.rpy* buscaremos la variable oneParameterEvents y agregaremos "dance", esta vez vinculando el nombre con la animación que hemos creado.

Vinculando el nombre "dance" a la animación Dance.



En este sentido, "dance" es el nombre que usaremos para llamar a la animación, mientras que "Dance" es el nombre que le dimos a la animación en el archivo *animations.rpy.*

Realmente también podríamos colocar la animación en la variable twoParameterEvents y seguiría funcionando.

De esta forma, para usar la animación en nuestros archivos de control, solamente tendríamos que escribir en la columna events.

personaje Sprite : dance

Nota: Debido al comportamiento del motor, si la animación utiliza la propiedad zoom, estrictamente debe estar en *twoParameterEvents.*

Balanceo de novela visual

Si tu novela visual es lineal y carece de decisiones y consecuencias puedes saltarte este módulo.

Lo ideal es que las decisiones de una novela visual influyan de una u otra manera sobre el futuro, esto ocasiona que pueda haber X cantidad de caminos posibles, por ejemplo, si nuestra novela visual tiene 2 decisiones de 2 opciones cada una, tendríamos un total de 4 caminos posibles.

Sin embargo, la cosa se puede complicar en gran medida pudiendo llegar a los millones de caminos posibles. Esto no es necesariamente un problema, al menos que nuestra novela tenga condiciones estrictas para alcanzar ciertos finales específicos.

Por ejemplo, en la novela visual **Tales of Venslla** (Actualmente en escritura) el protagonista puede ser expulsado numerosas veces si no cumple con ciertos puntos de calificación al final de cada periodo académico, cada una de esas expulsiones se traduce a un final.

Teniendo en cuenta que el final “bueno” de cada ruta requiere cumplir con cierta cantidad de puntos de amistad y de calificación, esto genera aún más finales posibles, si el sistema de puntos y decisiones no se balancea de manera correcta, va a ocasionar que el juego sea extremadamente difícil teniendo que escoger decisiones específicas para no terminar expulsado.

Es por esto que, el balanceo es importante y existe una herramienta desarrollada por mí cuyo nombre es **Visual Novel Technical Assistant (VNTA)**

En pocas palabras, **VNTA** es una herramienta que recibe las decisiones y consecuencias relevantes de tu novela visual, los analiza y te arroja resultados estadísticos sobre los finales posibles, pudiendo así, evaluar qué finales son más propensos a ocurrir y así definir qué tan fácil o difícil es nuestra novela visual.

```
Caminos posibles: 11210452
Endings:
   expulsado-2:
      Cantidad: 148
      Porcentaje: 0.00%
      Índice: 0.00

   expulsado-3:
      Cantidad: 37632
      Porcentaje: 0.34%
      Índice: 0.00

   expulsado-4:
      Cantidad: 254784
      Porcentaje: 2.27%
      Índice: 0.02

   expulsado-5:
      Cantidad: 3319808
      Porcentaje: 29.61%
      Índice: 0.30

   good:
      Cantidad: 4800870
      Porcentaje: 42.82%
      Índice: 0.43

   bad:
      Cantidad: 2797210
      Porcentaje: 24.95%
      Índice: 0.25
```

En esta imagen podemos apreciar los resultados estadísticos de la ruta de **Liz’Amar**, un personaje principal de **Tales of Venslla**.

Con un total de 11 millones de caminos posibles, la cantidad de rutas en la que el protagonista termina expulsado en el capítulo 2 y 3 son tan mínimas que son inferiores al 0.0%

A medida que avanzan los capítulos, el jugador es más propenso a cometer errores y se alcanza a ver cómo en el capítulo 5, el 29% de los caminos posibles terminan en una expulsión.

Pese a todo, el camino más común sigue siendo el final bueno con un 42%

Sin embargo, he de admitir que no fue así en un inicio, al usar por primera vez la herramienta más del 80% de todos los caminos posibles, terminaban en finales malos o expulsiones.

Es por esto que me vi en la necesidad de balancear la ruta de **Liz’Amar** jugando con los puntos de las decisiones hasta alcanzar un buen 42% de finales buenos.

Si estás interesado en probar esta herramienta te dejo el enlace al video de Youtube donde explico cómo usarla. https://www.youtube.com/watch?v=a2dJ_Ge2RAg

Derechos

Atlax Renpy es un motor de novelas visuales que utiliza como base el motor **Renpy**, cuya ventaja principal es poder crear novelas visuales con escasos conocimientos en programación e informática.

Cabe destacar que **Atlax Renpy** es una herramienta gratuita de código abierto que puede ser utilizada y modificada por cualquier persona sin requerir ningún tipo de permiso o aporte económico.

De igual manera, se aprecia en gran medida nombrar a **Atlax Renpy** en caso de que distribuyas tu novela visual usando este motor, de esa forma podrá llegar a oídos de otros usuarios que también puedan estar interesados.

Referencias

Cómo instalar Atlax Renpy:

Repositorio de Atlax Renpy: https://github.com/Atlantox/atlax-renpy

Github: https://github.com/

Códigos de colores hexadecimales: https://htmlcolorcodes.com/es/

Calculadora de ancho de pixeles de un texto: https://webutility.io/pixel-width-calculator

Repositorio con trducciones: https://github.com/Atlantox/renpy-languages

Modificadores de texto en Renpy: https://www.renpy.org/doc/html/text.html#text-tag-alpha

Visual Novel Technical Assistant: https://www.youtube.com/watch?v=a2dJ_Ge2RAg

Sitio web de Atlantox: https://atlantox.pythonanywhere.com/